Security Audit Report

XOXNO Lending Stellar

Delivered: TBD



Unofficial

Table of Contents

- XOXNO Lending
- Disclaimer
- Executive Summary
- Audit Scope & Goals
- Methodology
- Description of the System
  - Architecture
  - Shared code
  - Global invariants
- Pool
  - Pool — Description
  - Pool — Invariants
- Controller
  - Controller — Description
  - Controller — Liquidation
  - Controller — Flash positions
  - Controller — Invariants
- Price Aggregator
  - Price Aggregator — Description
  - Price Aggregator — Invariants
- Swap Aggregator
  - Swap Aggregator — Description
  - Swap Aggregator — Invariants
- Position NFT and XOXNO Oracle
  - Position NFT
  - XOXNO Oracle
  - Position NFT and XOXNO Oracle — Invariants
- Governance
  - Governance — Invariants
- DeFindex Strategy
  - DeFindex Strategy — Invariants
- Findings
  - Security findings
    - [S-01] A withdraw amount of zero closes the market and can leave an account at its borrowing limit
    - [S-02] LTV-only restamp on a loosened listing leaves a position's LTV and LT inconsistent, exposing borrowers to unexpected liquidation
    - [S-03] A borrowable market with a 100% utilization cap can be frozen until the pool is upgraded
    - [S-04] A market whose supply value passes about 1.7e11 whole units freezes on the next accrual

- Informative findings
  - I-01 Delegate grants survive an NFT transfer and reactivate when it returns
  - I-02 The optional `executor` argument and the EXECUTOR role restrict nothing
  - I-03 Endpoints take key-value vectors where a map would rule out duplicate keys
  - [I-04] Proposal salts separate pending operations; they do not prevent re-proposal
  - [I-05] Governance role-change events name the owner as the caller
  - [I-06] Blend pools and position managers can only be revoked through the timelock
  - [I-07] In-band sentinel values give ordinary numbers a second meaning
  - [I-08] The rate-model validation lets the cap flatten the curve below optimal utilization
- Future Work
- Appendix A: Automated analysis

XOXNO Lending

Table of Contents

The order below is the order of the rendered report. Nesting is heading depth. Adding a section means adding a file *and* a line here.

Findings are added here as they are written: one line per file in `12-findings/01-security/` or `12-findings/02-informative/`, nested under that section. Findings live in one chapter, not in the chapter that produced them.

Unofficial

Disclaimer

This report does not constitute legal or investment advice. You understand and agree that this report relates to new and emerging technologies and that there are significant risks inherent in using such technologies that cannot be completely protected against. While this report has been prepared based on data and information that has been provided by you or is otherwise publicly available, there are likely additional unknown risks. This report is also not comprehensive in scope, excluding a number of components critical to the correct operation of this system. This report is for informational purposes only and is provided on an 'as-is' basis. You acknowledge and agree that you are making use of this report and the information contained herein at your own risk. The preparers of this report make no representations or warranties of any kind, either express or implied, regarding the information in or the use of this report and shall not be liable to you or any third parties for any acts or omissions undertaken by you or any third parties based on the information contained herein.

Blockchain technology is still a nascent software arena, and any related implementation and public offering carries substantial risk.

Finally, the possibility of human error in the manual review process exists, and we recommend seeking multiple independent opinions on any claims which impact a large quantity of funds.

Unofficial

Executive Summary

XOXNO engaged Runtime Verification Inc. to conduct a security audit of XOXNO Lending, a collateralized lending protocol on Stellar Soroban, under the audit subsidy programme of the Stellar Development Foundation (SDF). The audit covers the eight contracts of the `XOXNO/rs-lending-xlm` repository at commit `a2486b25`, release `v1.1.0`: the pool, the controller, the price and swap aggregators, the position NFT, the XOXNO oracle, governance and the DeFindex strategy, together with the shared code they import. Its purpose is to establish whether the protocol keeps the assets it holds safe: that only entitled parties can move collateral and debt, that accounting and liquidation behave as designed, and that each privileged role is limited to its intended powers.

Two Runtime Verification auditors carried out the engagement between September 30 and November 13, 2026. The work proceeded in two phases. A design review built a model of the protocol from its documentation and code: the components and their interactions, custody of assets, roles and permissions, and the invariants the implementation must uphold, extending the list XOXNO had written itself. The code review then followed the order in which privilege flows through the system: governance first, then the controller and the pool, and the remaining components after them. Automated analysis ran alongside both phases: Specula, a model-based bug finder, in scans each focused on one area of the protocol, and Runtime Verification's Plainsight suite of security scanners over the repository.

Findings

Every finding in one line. The full write-ups, and the recommended action for each, are in Findings.

ID	Finding	Area	Found by	Severity	Difficulty	Status
S-01	A withdraw amount of zero closes the market and can leave an account at its borrowing limit	Controller	Code review	Low	High	Not Addressed
S-02	LTV-only restamp on a loosened listing leaves a position's LTV and LT inconsistent, exposing borrowers to unexpected liquidation	Controller	LLM-assisted code review	Low	Medium	Not Addressed
S-03	A borrowable market with a 100% utilization cap can be frozen until the pool is upgraded	Pool	LLM-assisted code review	Medium	High	Not Addressed
S-04	A market whose supply value passes about 1.7e11 whole units freezes on the next accrual	Pool, Controller	Code review	Low	High	Not Addressed
I-01	Delegate grants survive an NFT transfer and reactivate when it returns	Controller	Code review	—	—	Acknowledged
I-02	The optional `executor` argument and the EXECUTOR role restrict nothing	Governance	Code review	—	—	Not Addressed
I-03	Endpoints take key-value vectors where a map would rule out duplicate keys	Controller	Code review	—	—	Not Addressed
I-04	Proposal salts separate pending operations; they do not prevent re-proposal	Governance	Code review	—	—	Not Addressed
I-05	Governance role-change events name the owner as the caller	Governance	Code review	—	—	Not Addressed
I-06	Blend pools and position managers can only be revoked through the timelock	Governance, Controller	Code review	—	—	Not Addressed
I-07	In-band sentinel values give ordinary numbers a second meaning	Controller, Governance, Swap aggregator	Code review	—	—	Not Addressed

ID	Finding	Area	Found by	Severity	Difficulty	Status
I-08	The rate-model validation lets the cap flatten the curve below optimal utilization	Pool	Code review	—	—	Not Addressed

Remediation status

Unofficial

Audit Scope & Goals

Goals

The goal of the engagement is to establish that the XOXNO Lending contracts have no exploitable flaw or logic error that would let a party take, lock or devalue assets the protocol holds, and that the protocol is safe to operate as designed. Concretely, that only an account's owner or its delegate can move the account's collateral and debt; that supply, borrow, repayment, withdrawal and liquidation book exactly the amounts that move, with rounding in the protocol's favour; that a solvent account cannot be liquidated and an insolvent one cannot escape liquidation; that the prices the controller consumes are resolved as the oracle configuration specifies; and that every privileged role, from governance owner to guardian, is limited to the powers the design assigns it, with no shortcut around the timelock.

We pursued this on two fronts. We documented the protocol's components, their interactions and the invariants its implementation must uphold, so that the review measures the code against stated intent rather than against assumptions. We then reviewed the code for departures from those invariants, by reading it and with automated analysis, and reported each departure with its severity, the conditions under which it can be triggered and a recommendation. Findings that fall short of a security impact but would mislead users, operators or integrators are reported as informative.

Scope

The target code resides in the `XOXNO/rs-lending-xlm` repository.

Repository	Audited commit
`XOXNO/rs-lending-xlm`	`a2486b255b974ef8432ce44de014931f51552f4b` (`v1.1.0`)

In scope

Component	Location	Chapter
Shared code	`common/`	Description of the System
Pool	`contracts/pool/`	Pool
Controller	`contracts/controller/`	Controller
Price aggregator	`contracts/price-aggregator/`	Price Aggregator
Swap aggregator	`contracts/swap-aggregator/`	Swap Aggregator
Position NFT	`contracts/position-nft/`	Position NFT and XOXNO Oracle
XOXNO oracle	`contracts/xoxno-oracle/`	Position NFT and XOXNO Oracle
Governance	`contracts/governance/`	Governance
DeFindex strategy	`contracts/defindex-strategy/`	DeFindex Strategy

Out of scope

The audit covers only the smart contract code listed above. Third-party libraries, off-chain services, frontends, tests, scripts, CI configuration and formal verification specifications are excluded from the scope. The folders below are excluded on these grounds:

Excluded	Location
Certora verification libraries (patched third-party copies)	`vendor/`
Certora specifications	`certora/`
Off-chain services	`services/`
Tests and test harness	`tests/`
Mock contracts used by tests	`mock/`
Scripts	`scripts/`
CI workflows and GitHub configuration	`.github/`

Unofficial

Methodology

This chapter describes how the audit was conducted and every technique and tool used to conduct it. Where a component was reviewed differently, that deviation is recorded in that component's own chapter head rather than repeated here.

Runtime Verification followed a structured methodology to evaluate the correctness and security of the XOXNO Lending contracts within the agreed engagement timeline. Although manual review cannot guarantee the discovery of all vulnerabilities, our approach was designed to maximize coverage and identify the most impactful risks.

Engagement structure

The engagement was carried out by two Runtime Verification auditors over six and a half weeks, starting on September 30, 2026, and proceeded in two phases.

Design review and invariant identification. The engagement opened with a design review of the protocol, with the automated tools described below run on the codebase in parallel, so that their results were available as leads before the code review began. In the design review we built a model of the protocol: its architecture and the interactions between its contracts, its trust boundaries, the lifecycle of each entity it manages, how custody of assets is held and moves between contracts, and the roles across the protocol and the permissions each one holds. To contextualize the intended behavior, we used the documentation supplied by XOXNO, including its own list of invariants. This phase produced a set of functional and security invariants, extending XOXNO's list with properties it did not state. They are recorded per component in this report and guided the detailed review.

Code review. The code review started with the governance contract, through which privileged changes to the controller and the price aggregator are made, and continued with the controller, the entry point for supply, borrow, repayment, withdrawal and liquidation, and the contract that coordinates the pools, the price and swap aggregators and the position NFT.

Throughout the engagement we met with the XOXNO team twice a week. These meetings were used to clarify intended behavior, to discuss leads and findings as they emerged, and to agree on changes to the plan.

Techniques and tools

Every technique applied across the engagement is described here once. A chapter that used a technique differently, or did not use one at all, says so in its own head.

Tool output is evidence only once it records its version, configuration, target and date. Every run is logged in Automated analysis on those terms.

Manual code review

After establishing the protocol model, we performed a line-by-line review of the Soroban Rust codebase. This included examining:

- Correctness of supply, borrow and repayment accounting, and of interest accrual in the pools;
- Correctness of health-factor computation and of the liquidation logic in the controller;
- Authorization and permission boundaries, including the proposal, delay and execution flow of the governance contract;
- Price handling: oracle aggregation, staleness and the trust placed in each price source;
- Token handling, unit conversions and rounding direction in the mathematical logic;

- Edge cases, boundary conditions, and potential state inconsistencies, including across cross-contract calls;
- Non-functional properties such as performance, storage usage and TTL management;
- Threats to the protocol's availability, including denial of service.

Execution paths and state transitions were evaluated against the identified invariants to ensure alignment between the intended design and the implemented behavior. Where the intended behavior was ambiguous, we confirmed it with the XOXNO team, and recorded the agreed behavior in this report where the code or documentation leaves it implicit.

Finally, we held rounds of discussions within the audit team over the code and platform design to verify possible exploitation vectors and identify improvements for the analyzed contracts.

Specula

Specula is a tool for finding bugs in a codebase by primarily modeling it in a formal spec. Language-model agents build a TLA+ model of the behaviour under review, check the model against execution traces from an instrumented test harness, and run the TLC model checker over it in search of invariant violations. Every candidate the model checker or the agents' code review raises is then confirmed or dismissed by writing a reproduction test against the unmodified contracts, and the confirmed ones are rated. We ran Specula 1.2.0 at the audited commit, once per focus, each run steered by a guidance document that sets its goal, scope and the questions to answer.

The runs are recorded in Automated analysis; their full outputs were supplied to XOXNO.

Each run focused on one area of the protocol:

- Scan 1, October 1 to 2: takeover of a contract's owner or admin powers, abuse of the lower governance roles and the timelock lifecycle, code the controller invokes without trusting it, and manipulation of the prices the controller consumes. 22 candidates, 6 reproduced.
- Scan 2, October 15 to 16: custody of the pool and controller balances and of the value in lending accounts: callers acting on accounts they are not entitled to, token flows between caller, controller and pool, caller-supplied contract addresses, unusual composition of operations, and the lower-privileged roles and permissionless maintenance entry points. 27 candidates, 4 reproduced.
- Scan 3: pending.
- Scan 4: pending.
- Scan 5: pending.

A candidate is a lead until its reproduction runs, and a reproduction shows the behaviour in a local test environment built from the audited commit; exposure on mainnet depends on the deployed configuration. The model abstracts the code, so a clean model-checking run is evidence of coverage, not proof of absence. Specula's severity ratings are its own: every reproduced candidate was reviewed by us before it became a finding, and the ratings in this report are ours.

All reports have been shared with the client and the findings have been clarified as either false positives (not harmful or design intended) or valid where applied.

Plainsight

Plainsight runs a set of off-the-shelf security scanners over a repository and collects their results into a single report. We ran it once, at the audited commit, with nine scanners enabled:

- dependency vulnerabilities: `osv-scanner`, `grype` and `trivy`;
- leaked secrets: `gitleaks` and `trufflehog`;

- static analysis: `semgrep` and `bandit`;
- CI and infrastructure configuration: `zizmor` and `checkov`.

The run produced 1,150 results. We used AI-assisted triage to filter out false positives and duplicates, which left about 90 that pointed at real issues. Most of them were in dependencies of the off-chain services, the test SDK and the CI tooling, or in the configuration of the GitHub Actions workflows. Two advisories, in `anyhow` and `paste`, are in build-time dependencies of the contract crates, which are not part of the deployed WASM. None concerned the deployed contracts. Third-party libraries, the off-chain services, the tests and the CI configuration are outside the scope of this audit, so the results were passed on to XOXNO without further validation and are not reported as findings.

Plainsight's scanners match known patterns and published advisories; they do not reason about contract logic, and its results are no evidence for or against the correctness of the contracts.

AI-assisted review

We used large language models (Claude) to support the manual review, not to replace it. They were used to produce an initial overview of the contracts, their interactions and interfaces at the start of the engagement; to help triage the output of the automated tools; and to work from transcripts of recorded client meetings when writing up the agreed intended behavior.

Finding ratings

Findings carry two independent ratings.

Severity: impact if exploited: High, Medium, Low.

Difficulty: how hard to exploit: Low, Medium, High. *Low* is the worse one.

Each finding also carries a recommended action (Fix Design, Fix Code, or Document Prominently) and a status: Not Addressed, Partially, Addressed, or Acknowledged.

Informative findings carry neither rating.

A finding's ID is `S` or `I`, for security or informative, then its index: `S-01`, `I-03`. How a finding was found is recorded in the finding, not in its ID.

All findings, of both kinds, are collected in Findings. The chapters name their findings and link to them; the finding itself is written once.

Reporting

Findings were reported to XOXNO as they were confirmed, through a shared Telegram group and in the meetings described above. Where a potential finding depended on the intended behavior, it was discussed with XOXNO before being confirmed or dismissed. Where the automated tools flagged issues in code outside the audit scope, such as dependency advisories in the off-chain services, these were passed on to XOXNO without validation.

Description of the System

XOXNO Lending is a collateralized lending protocol on Stellar Soroban. Suppliers earn interest on borrowed liquidity. Borrowers hold debt against collateral, with account risk enforced by a controller and tokens held in a central pool.

Core concepts

- A **hub** groups markets. A **market** is one token in one hub; it has its own balances, interest rate and indexes. The same token in two hubs is two markets.
- A **spoke** is a set of risk rules: which markets an account may use, and with what loan-to-value, liquidation threshold, bonus and caps.
- An **account** holds supply and debt positions. It is bound to one spoke for life and is represented by a position NFT; whoever holds the NFT controls the account.

The client describes the same model in `docs/reference/architecture.md`.

Components

Component	Purpose	Chapter
Controller	The entry point for lending: accounts, risk checks, liquidation, strategies	Controller
Pool	Holds all tokens and each market's books; only the controller can change it	Pool
Price aggregator	Serves validated USD prices to the controller	Price Aggregator
Swap aggregator	DEX router, used by the controller's strategies and callable directly	Swap Aggregator
Position NFT	One token per account	Position NFT
XOXNO oracle	Price feed set by a quorum of signers	XOXNO Oracle
Governance	Timelocked administration of the controller and price aggregator	Governance
DeFindex strategy	Lets a DeFindex vault supply to one market	DeFindex Strategy
`common/`	Math, interest rates, oracle helpers and types shared by all contracts	Shared code

Participants

Participants are the parties that use the protocol. They are distinct from *governance roles* (PROPOSER, EXECUTOR, CANCELLER, GUARDIAN, ORACLE), which are on-chain permissions held by addresses in the governance contract; see Governance.

Participant	Expected use
Supplier / borrower	Supplies collateral, borrows, repays and withdraws through the controller
Leverage user	Opens and adjusts leveraged accounts with the controller's strategies
Position manager	A contract or bot registered by governance; acts on accounts whose holders added it as a delegate
Liquidator	Repays debt of unhealthy accounts in exchange for collateral plus a bonus
Keeper	Calls maintenance functions anyone may call: interest accrual, revenue claims, storage renewal
DeFindex vault	Deposits into and withdraws from one market through the strategy adapter
Governance owner and governance role holders	Configure markets and risk, upgrade contracts, respond to emergencies
Router admin	Runs the swap aggregator: fees, referrals, upgrades
Oracle admin and signers	Run the XOXNO oracle: the admin manages signers and feeds, signers submit prices

Permissions at a glance

Who can perform each kind of action, and how. A *proposal* is a governance change that runs only after the delay described in Governance; an *immediate* action skips that delay.

Who	Action	How
Account holder	Add or remove delegates	Direct call
Account holder or delegate	Borrow, withdraw or run strategies on a lending account	Direct call
Anyone	Execute a ready proposal	After the delay
Anyone	Supply, repay, liquidate, or call maintenance functions	Direct call
CANCELLER	Cancel a pending proposal	Before it runs
Governance owner	Upgrade the controller, pool, position NFT, price aggregator or governance	Proposal
Governance owner	Change price sources or tolerances	Proposal
Governance owner	Change the swap aggregator, price aggregator or revenue address	Proposal
Governance owner	Approve a Blend pool for migration	Proposal
Governance owner	Grant governance roles, transfer ownership, lengthen the delay	Proposal
Governance owner	Remove a GUARDIAN or ORACLE role	Immediate
GUARDIAN	Pause the protocol, or add restrictions to a listing	Immediate
GUARDIAN, or PROPOSER	Create hubs and spokes	Immediate, or proposal
ORACLE	Narrow a price's allowed range	Immediate
Oracle admin	Manage oracle signers, thresholds and feeds, upgrade the oracle	Direct call
PROPOSER	Revoke a governance role (not the owner's or one's own)	Proposal
PROPOSER	Create markets, list assets, set risk parameters, caps and limits	Proposal
PROPOSER	Unpause the protocol, or remove restrictions from a listing	Proposal
PROPOSER	Register or deactivate a position manager	Proposal
PROPOSER	Force a bad-debt write-off	Proposal
Registered signer	Submit oracle prices	Direct call
Router admin	Set router fees and referrals, sweep, upgrade the router	Direct call

The governance owner always holds every governance role, so it can also do everything listed for PROPOSER, CANCELLER, GUARDIAN and ORACLE.

Trust model

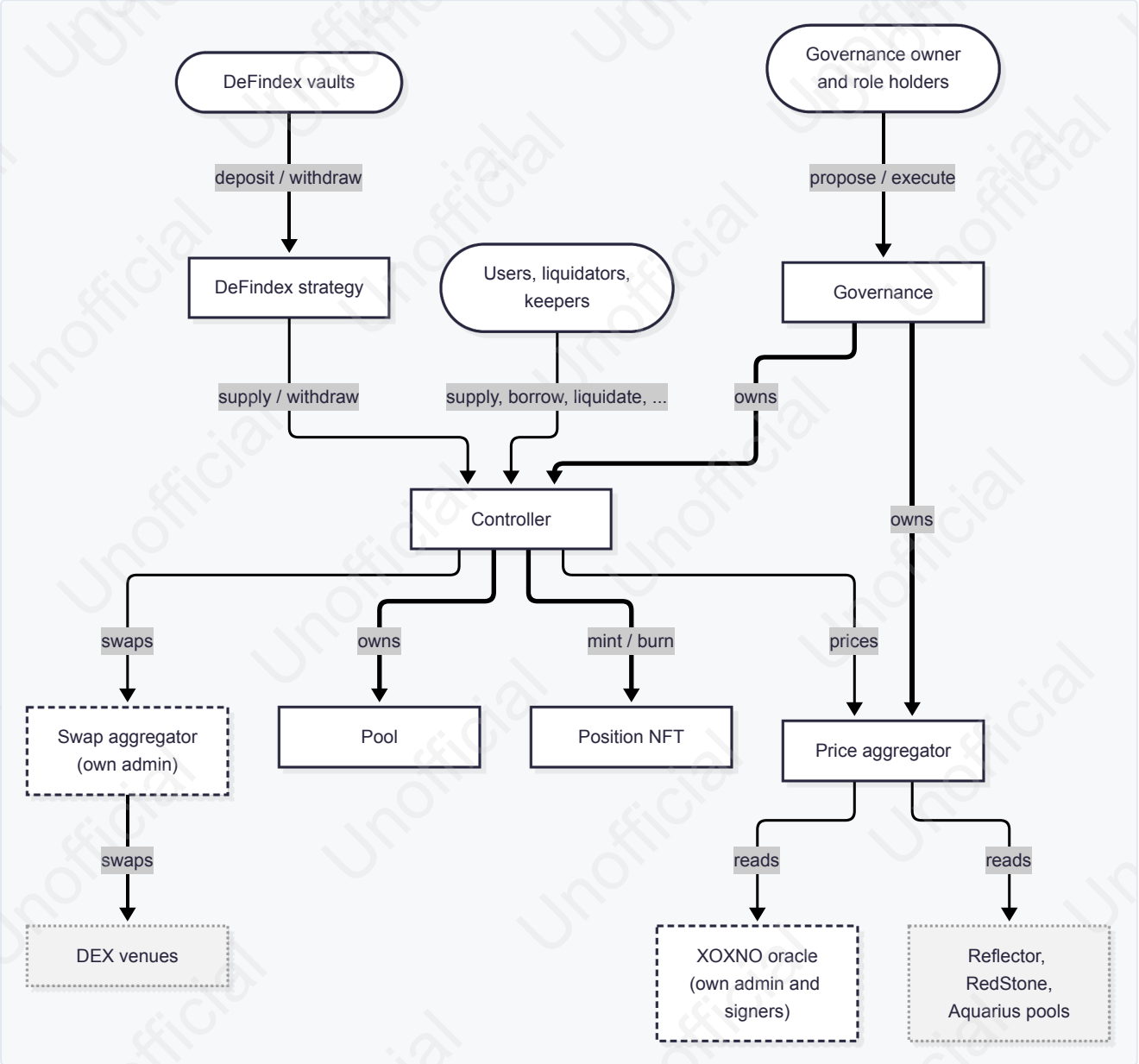
The protocol trusts the governance owner, the two standalone admins (router and XOXNO oracle), and the configured price sources within the aggregator's checks. It treats listed tokens, DEX venues, flash-loan receivers and Blend pools as untrusted: it measures balances instead of relying on what they report, and authorizes them for exactly one transfer at a time. The client's `docs/explanation/threat-model.md` gives the full list.

The swap aggregator and the XOXNO oracle sit outside lending governance: their admins can change or upgrade them without a governance proposal.

Architecture

Contracts and calls

Users lend and borrow only through the controller. The controller owns the pool and the position NFT, reads prices from the price aggregator, and sends swaps through the swap aggregator. Governance owns the controller and the price aggregator. The swap aggregator and the XOXNO oracle have their own admins.



Thick arrows are ownership; thin arrows are calls. Dashed boxes are run by their own admins, outside lending governance; grey boxes are external. The price aggregator and the swap aggregator never call back into the controller.

Deployment and ownership

Every contract sets its owner in its constructor; none has a separate initialization step.

1. Governance is deployed. Its deployer becomes owner and holds every governance role.
2. Governance deploys the controller, which starts paused.
3. Governance deploys the price aggregator and points the controller at it.
4. Through governance proposals, the controller deploys the pool and the position NFT, with itself as their owner.
5. The swap aggregator and the XOXNO oracle are deployed separately by their own admins. Governance then points the controller at the router.
6. Governance configures hubs, markets, prices and spokes, then unpauses.

Custody

The pool holds every token. When tokens come in, the controller transfers them to the pool, measures what actually arrived, and tells the pool to credit that amount. When tokens go out, the pool pays them on the controller's instruction.

Account lifecycle

Here an *account* is a lending account: a record kept by the controller that holds one set of supply and debt positions, identified by a numeric id. It is not a Stellar address. One address can hold several lending accounts, each represented by its own position NFT.

1. **Create.** A user calls `supply` (or opens a leveraged position) with account id 0. The controller creates a new lending account and mints its NFT to the caller's address.
2. **Use.** The NFT holder supplies, borrows, withdraws and repays against that lending account. Borrowing and withdrawing must leave it healthy.
3. **Liquidate.** If the lending account's health factor falls below 1, anyone can liquidate it.
4. **Close.** When the lending account holds no positions, the controller deletes it and burns its NFT. A lending account left with dust collateral and unpaid debt can be cleaned up as bad debt.

Authorization

- **Holder.** The current NFT holder controls the account. Selling or transferring the NFT transfers the whole position.
- **Delegates.** A holder can let up to 16 registered position managers act on the account. A grant only works while the holder who made it still holds the NFT.
- **Outside code.** When the controller calls a router, callback or Blend pool, it authorizes exactly one token transfer and blocks its own money-moving functions until the call returns.
- **Administration.** Controller settings change only through governance, and remain available while the protocol is paused.

Documentation and code

- The client's architecture document says the pool "books the smaller measured amount" when a token under-delivers. The measuring is done by the controller; the pool credits what it is told.
- The same document implies governance's deploy helpers can run in either order. Only the price-aggregator helper wires the two contracts together, so the controller must be deployed first (`governance/src/deploy.rs`).

Shared code

`common/` is a Rust library in the audited repository that every contract is compiled with. It is not a deployed contract and holds no state of its own.

Module	What it provides
`math/`	Fixed-point number types: `Wad` (18 decimals) for USD values and health factors, `Ray` (27 decimals) for shares, indexes and rates, `Bps` (10,000 = 100%) for percentages
`rates/`	The interest model: a three-segment rate curve over utilization, compounding, and the indexes that turn shares into amounts; see Pool — How interest works
`oracle/`	Price helpers used by the price aggregator: normalizing provider prices, LP-token fair value, and client interfaces for Reflector, RedStone, Aquarius and the XOXNO oracle
`types/`	The data passed between contracts, such as market keys, positions, listings and oracle configurations
`token.rs`	A transfer that returns the amount actually received, and a one-time transfer authorization for outside contracts
`validation.rs`, `errors.rs`, `constants/`	Shared parameter checks, error codes and limits
`ttl.rs`, `collections.rs`	Storage-lifetime renewal and small collection helpers

Global invariants

These properties span several contracts. Each is phrased so that a single counterexample would break it. The component chapters list their own invariants and refer back here where a component can break a global one. An invariant taken from the client's documentation says so in its source note. Whether the implementation holds each one is part of the review; violations are reported as findings.

Authority and consent

INV-SYS-01 — One chain of authority. Only governance can change the controller's or the price aggregator's configuration, code or owner, and only the controller can change the pool or mint and burn position NFTs. No other address or contract can make these changes. *Source: client documentation.*

INV-SYS-02 — Moving an account's value needs its holder's consent. Apart from interest accrual, debt can be added to a lending account, or collateral taken out of it, only with the signature of its NFT holder or of a delegate that holder approved. The only exceptions are liquidating an unhealthy account and writing off an insolvent one. *Source: client documentation.*

INV-SYS-03 — Emergency powers only restrict. After deployment, every action that skips the governance delay can only restrict the protocol (pause, add listing flags, narrow a price range, remove an emergency role) or create empty structures (hubs, spokes). Anything that loosens a restriction or moves funds waits for the delay. *Source: client documentation.*

Funds and accounting

INV-SYS-04 — Tokens leave the pool only for an entitled recipient. The pool pays out tokens only to: a borrower or withdrawer (or the recipient they name), a liquidator, a flash-loan receiver that returns the loan in the same transaction, a payer receiving their own excess back, or the controller itself, which uses them in a strategy or forwards claimed revenue to the configured revenue address.

INV-SYS-05 — Account records match the pool. For every market, the supply shares of all lending accounts plus the protocol's revenue shares equal the pool's total supply shares, and the debt shares of all lending accounts equal the pool's total debt shares. *Source: client documentation.*

INV-SYS-06 — Suppliers lose value only through an explicit write-off. The value of one supply share never decreases, except when bad debt in that market is written off. *Source: client documentation.*

Risk and prices

INV-SYS-07 — Risk-increasing actions end healthy. Any borrow, withdrawal or strategy that leaves debt on a lending account leaves that account healthy at current validated prices. *Source: client documentation.*

INV-SYS-08 — No valuation uses an unsafe price. Every risk check, liquidation and write-off uses only prices the price aggregator accepts. If any needed price is missing, stale or out of range, the operation reverts. *Source: client documentation.*

Availability and external calls

INV-SYS-09 — Pause stops new risk but not exits. While the protocol is paused, nothing can add risk (supply, borrow, strategies, flash loans), and holders can still withdraw and repay, liquidators can still liquidate, and bad debt can still be cleaned up. *Source: client documentation.*

INV-SYS-10 — Outside code cannot re-enter lending. While the controller has handed control to outside code (a flash-loan receiver, a callback, the router, a Blend pool), no function of the controller that moves funds can run. *Source: client documentation.*

Pool

The pool (`contracts/pool/`) holds every token in the protocol and keeps the books of every market. It does not check users or risk: its owner, the controller, decides what happens, and the pool does the arithmetic and moves the tokens.

Pool — Description

How it works

Each market keeps a cash balance, supply and debt shares, interest indexes, protocol revenue and a rate model. Every change follows the same steps: load the market, accrue interest up to now, apply the change, check the market's limits (enough cash, utilization under its maximum), save, and only then send tokens out.

The pool trusts the controller about incoming amounts. For supply, repayment and recapitalization, the controller transfers the tokens in first and tells the pool how much arrived; the pool credits that amount.

How interest works

Positions are stored as shares. Each market keeps a supply index and a borrow index; a position's amount is its shares times the index. When time passes, the pool raises the borrow index according to the market's rate curve, raises the supply index by the suppliers' share of that interest, and books the rest as protocol revenue. Lending accounts never need to be updated one by one.

The client's `docs/reference/formulas.md` gives the exact formulas and rounding rules.

Interface

All functions that change state can be called only by the pool's owner, the controller (`contracts/pool/src/lib.rs`).

Function	Purpose
`supply` , `withdraw`	Add or remove supply shares; `withdraw` pays the receiver
`borrow` , `repay`	Add or remove debt; `borrow` pays the receiver, `repay` refunds overpayment
`net_settle`	Cancel supply against debt in the same market, with no tokens moving
`seize_positions`	Write off bad debt against suppliers, or turn seized shares into revenue
`flash_loan`	Lend tokens to a receiver contract for one call and collect them back with a fee
`create_strategy`	Open debt for the controller's leverage strategies, with or without a fee
`recapitalize`	Accept tokens that cover a market's shortfall, refunding any excess
`claim_revenue`	Pay accumulated protocol revenue to the controller
`create_market` , `update_params` , `update_indexes`	Create a market, change its rate model, accrue interest
`upgrade`	Replace the pool's code

Read-only views (rates, utilization, reserves, supplied and borrowed totals, indexes) are open to anyone.

Interactions

- **Controller:** the only caller of state-changing functions. It deploys the pool with itself as owner, and the pool cannot change owner.
- **Flash-loan receivers:** called back during `flash_loan`; the pool checks its own balance afterwards to make sure principal and fee came back.
- **Tokens:** the pool holds them all. The same token listed in two hubs shares one balance but has separate books.

Usage by participant

Users never call the pool directly. Suppliers, borrowers and liquidators act through the controller; governance changes markets through the controller's admin functions.

Documentation and code

- The pool's README says `update_indexes` writes only if time has passed. The code always saves the market, by design (comment in `ops/market.rs`).

Pool — Invariants

The pool can also break global INV-SYS-04, INV-SYS-05 and INV-SYS-06.

Authority

INV-POOL-01 — Only the controller changes the pool. Every function that changes balances, shares, market parameters or code fails unless called by the pool's owner, the controller, and the owner cannot be changed. *Source: client documentation.*

Cash and shares

INV-POOL-02 — Booked cash is backed by tokens held. After every operation, for each token, the cash booked across all of its markets never exceeds the pool's actual balance of that token. Tokens sent to the pool outside an operation are not counted as cash. *Source: client documentation.*

INV-POOL-03 — Value never moves without shares. A positive supply or borrow always creates shares, and a positive withdrawal or repayment always burns shares. *Source: client documentation.*

INV-POOL-04 — Share totals stay consistent. Supply, debt and revenue share totals are never negative, and revenue shares are always part of, and never more than, total supply shares. *Source: client documentation.*

INV-POOL-05 — Markets are isolated. An operation changes only the markets it names. Two markets for the same token in different hubs never share balances, shares or indexes; they share only the physical token balance.

Interest

INV-POOL-06 — Indexes move in the expected direction. The borrow index never decreases. The supply index decreases only when bad debt is written off. Both stay within the protocol's fixed bounds. *Source: client documentation.*

INV-POOL-07 — Interest is fully accounted for. The interest borrowers are charged is split between suppliers and protocol revenue. Rounding never credits suppliers and revenue together with more than borrowers were charged. *Source: client documentation.*

Liquidity

INV-POOL-08 — Liquidity limits hold. A borrow or an ordinary withdrawal cannot push a market's utilization above its configured maximum, and a borrow must leave cash of at least 2% of the market's supply. Liquidation withdrawals skip the utilization limit. *Source: client documentation.*

INV-POOL-09 — No debt without supply. No withdrawal, settlement or revenue claim can leave a market with outstanding debt and no supply. *Source: client documentation.*

INV-POOL-10 — Flash loans return exactly. After a flash loan the pool's token balance is higher by exactly the fee, and no debt was created. Otherwise the whole transaction reverts. *Source: client documentation.*

Revenue and shortfalls

INV-POOL-11 — Revenue payouts are bounded. A revenue claim never pays more than the revenue booked for that market or the cash it holds, and it pays only the controller. *Source: client documentation.*

INV-POOL-12 — Recapitalization only fills a shortfall. Recapitalization never creates shares, never credits more than the market's backing shortfall, and refunds the rest to the payer. *Source: client documentation.*

Unofficial

Controller

The controller (`contracts/controller/`) is the only contract users call to lend and borrow. It keeps every account's positions, decides who may act on an account, checks risk with prices from the price aggregator, runs liquidations and leverage strategies, and instructs the pool to move tokens. Its owner is governance.

Controller — Description

How it works

An account belongs to one spoke and has a mode: **Normal** for plain lending, or **Multiply**, **Long** or **Short** for leveraged positions opened by a strategy. The holder of the account's NFT controls it, and can add registered position managers as delegates.

Every call that moves money checks the caller's signature. Calls that add risk (supply, borrow, strategies, flash loans) are blocked while the protocol is paused; calls that reduce risk (withdraw, repay, liquidation, bad-debt cleanup) stay open. Each listing also has its own flags: **paused** blocks entry and exit, **frozen** blocks new entry, and **no-seize** protects it from liquidation.

After a borrow, a withdrawal or a strategy, the account must stay healthy: its loan-to-value-weighted collateral must cover its debt, its health factor must be at least 1, and an account with debt must keep a minimum amount of collateral ($5 by default). Supply and repayment skip these checks. Prices come from the price aggregator once per call; if any price is missing or unsafe, the call fails.

Interface

The entry points are in `contracts/controller/src/lib.rs` .

Accounts and positions

Function	Caller	Purpose
`supply`	anyone	Deposit collateral; account id 0 creates a new account. Others may only top up assets the account already holds
`borrow`	holder or delegate	Take debt and receive the tokens
`withdraw`	holder or delegate	Take collateral out; 0 means everything
`repay`	anyone	Pay down an account's debt
`add_delegate` , `remove_delegate`	holder	Let a registered position manager act on the account, or stop it
`renew_account`	holder	Extend the account's storage lifetime

Strategies (one call, several steps)

Function	Caller	Purpose
`multiply`	holder or delegate	Borrow, swap to collateral and supply: opens leverage
`swap_debt`	holder or delegate	Replace one debt token with another
`swap_collateral`	holder or delegate	Replace one collateral token with another
`repay_debt_with_collateral`	holder or delegate	Use collateral to pay debt, optionally closing the position
`flash_position`	holder or delegate	Open leverage through a custom callback contract; see Flash positions
`migrate_from_blend`	Blend user	Move a position from an approved Blend pool
`flash_loan`	anyone	Classic flash loan of pool liquidity

Liquidation and maintenance (anyone): `liquidate`, `clean_bad_debt` (see Liquidation), `update_indexes`, `claim_revenue` (pays the configured revenue address), `update_account_threshold` (applies new risk parameters to existing accounts), and `recapitalize` (covers a market's shortfall).

Views (anyone): health factor, collateral and debt values, positions, liquidation estimates, market indexes, spoke and listing settings.

Administration (governance only): set the price aggregator, swap aggregator and revenue address; deploy the pool and NFT; create hubs, spokes and markets; list assets and set their risk parameters and flags; register position managers and Blend pools; pause and unpause; upgrade; and force bad-debt cleanup.

Interactions

- **Pool:** every token movement and every change to a market's books.
- **Position NFT:** minted when an account is created, burned when it closes, and read to find each account's owner.
- **Price aggregator:** prices for every risk check and liquidation.
- **Swap aggregator:** the swaps inside `multiply`, `swap_debt`, `swap_collateral` and `repay_debt_with_collateral`. The user supplies the route; the controller authorizes one transfer of the input and checks what comes back.
- **Governance:** the owner; every admin function is called through it.

Usage by participant

- **Suppliers and borrowers** call `supply` to open an account, then `borrow`, `repay` and `withdraw`.
- **Leverage users** open and adjust positions with the strategies, and close them with `repay_debt_with_collateral`.
- **Position managers** act on accounts whose holders added them as delegates.
- **Liquidators** watch health factors and call `liquidate`.
- **Keepers** call the maintenance functions.
- **Governance** configures markets and risk, and responds to emergencies.

Documentation and code

- The client's endpoint reference omits one post-operation check: collateral in tokens with fewer than 3 decimals must be held in whole units (`risk/validation.rs`).
- The documentation describes supply for Normal accounts only. The code also accepts plain supply into Multiply, Long and Short accounts.

Controller — Liquidation

When an account's health factor falls below 1, anyone can liquidate it. The liquidator repays part or all of the account's debt and receives the matching value of its collateral plus a bonus. Debt that the collateral cannot cover is written off against the suppliers of the affected markets.

The client documents the formulas in `docs/reference/formulas.md`; the code is in `contracts/controller/src/positions/liquidation/`.

How it works

1. **Admission.** The account must have debt and a health factor below 1.
2. **Sizing.** The controller computes how much debt to repay: by default, enough to bring the account back to the spoke's target health factor. The bonus grows as the health factor falls, following a curve set per spoke, and is capped so that liquidation cannot make the account less healthy.
3. **Repayment.** The liquidator's debt tokens go to the pool. Any excess is refunded.
4. **Seizure.** Collateral is taken from every collateral position in proportion to its value; the liquidator cannot choose. The protocol keeps a fee on the bonus part.
5. **Payout.** The liquidator receives either:
   - **Transfer:** collateral tokens; or
   - **Credit:** supply shares moved into another Normal account on the same spoke that the liquidator controls (or a new one). No tokens leave the pool.

Liquidation stays available while the protocol is paused. A listing marked **no-seize** cannot be seized, and a **paused** listing's debt cannot be repaid.

Bad debt

If an account's debt exceeds its collateral, the remainder is written off: the remaining collateral becomes protocol revenue and the debt is removed by lowering the supply index of the debt markets, so all suppliers share the loss.

- `clean_bad_debt` (anyone): allowed when the collateral left is worth $5 or less. This threshold is fixed in the code.
- `force_socialize_bad_debt` (governance): allowed for any insolvent account; the client documents it as an irreversible step with its own runbook.

Interface

Function	Caller	Purpose
`liquidate`	anyone	Repay debt and seize collateral, as tokens or credited shares
`get_liquidation_estimate`	anyone (view)	Preview a liquidation
`clean_bad_debt`	anyone	Write off a dust-collateral insolvent account
`force_socialize_bad_debt`	governance	Write off any insolvent account

Usage by participant

- **Liquidators** monitor health factors, preview with the estimate view, and call `liquidate`.

- **Keepers** call `clean_bad_debt` on insolvent accounts with dust collateral.
- **Governance** uses forced cleanup for larger insolvent accounts.

Documentation and code

- The client's documentation lists ownership and position limits as the only rules for a Credit receiver. The code also requires whole units for collateral tokens with fewer than 3 decimals.

Controller — Flash positions

A flash position opens a leveraged account in one call, using the caller's own contract to source the collateral. It is the callback-based alternative to `multiply`, which uses the swap aggregator instead. The code is in `strategies/flash_position.rs`.

How it works

1. The caller names a debt market, an amount, a receiver contract, and the collateral it promises to deliver with a minimum for each.
2. The controller creates the debt without a fee and sends the borrowed tokens to the receiver.
3. The receiver's `execute_flash_position` callback runs. It must deliver the promised collateral to the controller, for example by swapping the borrowed tokens.
4. The controller measures what arrived, supplies it to the account, refunds any listed leftover tokens to the caller, and checks that the account is healthy.

The account must be Multiply, Long or Short, and the debt market must allow flash loans. While the callback runs, the controller blocks its own money-moving functions. If anything fails, the whole call reverts.

Flash position and flash loan

	Flash position	Flash loan
Result	An open, healthy leveraged account	Nothing left behind
Fee	None	Market flash-loan fee
Repayment	Collateral delivered by the callback	Principal plus fee pulled back by the pool
Callback	`execute_flash_position`	`execute_flash_loan`

Usage by participant

Leverage users and position managers who want custom routing (for example, through venues the swap aggregator does not support) deploy their own receiver contract and call `flash_position`.

Documentation and code

- The client lists four conditions on the declared collateral. The code also applies the normal supply checks (listed, not paused or frozen, usable as collateral) and the account's position limits before any token moves.

Controller — Invariants

The controller can also break global INV-SYS-01, INV-SYS-05, INV-SYS-08, INV-SYS-09 and INV-SYS-10.

Accounts and authority

INV-CTRL-01 — Only the holder or an approved delegate acts on an account. Borrowing, withdrawing and running a strategy on an existing lending account succeed only if signed by the current NFT holder, or by a position manager that is active and that the current holder added as a delegate. Only the holder can add or remove delegates. Classic flash loans, which create no position, are open to anyone. *Source: client documentation.*

INV-CTRL-02 — Others can only help an account. Anyone may repay a lending account's debt or add more of an asset it already supplies, but no one other than the holder or a delegate can open a new position in it. *Source: client documentation.*

INV-CTRL-03 — An account's spoke and mode never change. A lending account keeps the spoke and mode it was created with. Every later operation on it uses that spoke's rules. *Source: client documentation.*

INV-CTRL-04 — Accounts and NFTs exist together. Every lending account has exactly one position NFT with the same id, and the two are created and removed in the same transaction. *Source: client documentation.*

Accounting

INV-CTRL-05 — Credit matches what arrived. When tokens come in (supply, repayment, recapitalization, strategies), the controller credits the amount that actually reached the pool, never the amount requested. *Source: client documentation.*

INV-CTRL-06 — Revenue goes only to the configured address. Revenue claimed from the pool is forwarded only to the revenue address set by governance.

Risk

INV-CTRL-07 — Risk-increasing actions end healthy. After a borrow, a withdrawal or a strategy, an account with debt has a health factor of at least 1, loan-to-value-weighted collateral covering its debt, and at least the minimum collateral amount. Otherwise the transaction reverts. *Source: client documentation.*

INV-CTRL-08 — Valuations err on the safe side. In risk checks, collateral is rounded down and debt is rounded up, so rounding can never make an account look healthier than it is. *Source: client documentation.*

INV-CTRL-09 — Limits hold. Supply and borrowing respect the market's caps and the per-account position limits, and an account never has more than 16 delegates. Collateral credited to a liquidator's account in a Credit-mode liquidation is exempt from caps. *Source: client documentation.*

INV-CTRL-10 — Listing flags are respected. A frozen listing accepts no new supply or borrowing; a paused listing accepts no entry or exit; a no-seize listing is never seized in a liquidation. *Source: client documentation.*

INV-CTRL-11 — Listings are coherent. Every listing has a loan-to-value below its liquidation threshold, and a bonus small enough that the threshold times one plus the bonus is at most 100%. *Source: client documentation.*

Liquidation and bad debt

INV-CTRL-12 — Only unhealthy accounts are liquidated. A liquidation succeeds only on an account with debt and a health factor below 1. *Source: client documentation.*

INV-CTRL-13 — Liquidation does not make an account worse. Liquidating an account that still has more collateral than debt never lowers its health factor.

INV-CTRL-14 — Seizure matches repayment. A liquidator receives collateral worth at most the debt actually repaid times one plus the bonus, taken from every collateral position in proportion to its value. If fewer debt tokens arrive than planned, the seizure shrinks in the same proportion. The one exception is collateral in a token with fewer than 3 decimals, which is seized in whole units and can pay slightly more. *Source: client documentation.*

INV-CTRL-15 — Write-offs happen only to insolvent accounts. Bad debt is written off only for an account whose debt exceeds its collateral. Anyone can trigger it only when the remaining collateral is worth $5 or less; any larger write-off needs governance. *Source: client documentation.*

Strategies and outside code

INV-CTRL-16 — Outside contracts move only what was authorized. When the controller calls the router, a callback or a Blend pool, that contract can move only the tokens the controller sent it or the transfers it explicitly authorized for that call, and nothing else the controller holds. *Source: client documentation.*

INV-CTRL-17 — Strategies settle on measured amounts. A strategy credits only the swap output it measured, and returns unused input to the caller. *Source: client documentation.*

INV-CTRL-18 — A flash position leaves an open account. A flash position ends with an open, healthy account that still holds the new debt and at least one collateral position. *Source: client documentation.*

Administration

INV-CTRL-19 — Flag changes outside the relax path only tighten. Editing a listing or setting its flags can only add restrictions. Removing one requires the dedicated relax operation, and only against the flag state it was proposed for. *Source: client documentation.*

Price Aggregator

The price aggregator (`contracts/price-aggregator/`) gives the controller one validated USD price per asset. It reads the configured price sources at the moment it is asked, combines them, and refuses to return a price that is stale, inconsistent or outside its expected range. Its owner is governance.

Price Aggregator — Description

How it works

Each asset has a configuration with one or two price sources. Nothing pushes prices into the aggregator; it reads its sources each time a price is requested. A source can be:

- **A feed:** Reflector (latest price, or an average of recent records), RedStone, or the XOXNO oracle.
- **A scaled price:** a factor feed multiplied by another asset's price, for example an exchange rate times a base price.
- **An LP token price:** the fair value of an Aquarius pool share, computed from the pool's reserves and the prices of its two tokens rather than from its trading price.

With two sources, they must agree within a set tolerance, and the price used is their midpoint. If either source is missing or stale, the price fails; there is no fallback to one source. A configuration that relies only on raw spot market prices is rejected when it is set.

Every price is then checked for freshness and against a sanity band, a minimum and maximum value set per asset. A price that fails any check is never returned to the controller.

Interface

The entry points are in `contracts/price-aggregator/src/lib.rs` .

Function	Caller	Purpose
`prices`	anyone	Validated prices; fails if any requested price is unusable
`quotes`	anyone	The same prices with a status for each, instead of failing
`price_spread`	anyone	The two sources' prices for one asset
`oracle`	anyone	An asset's configuration
`set_oracle`	owner (governance)	Set or replace an asset's sources and limits
`set_tolerance`	owner	Change how closely two sources must agree
`set_sanity_band`	owner	Narrow an asset's allowed price range
`upgrade`	owner	Replace the code

Interactions

- **Controller:** calls `prices` for every risk check and liquidation, and `quotes` for one informational view.
- **Price sources:** Reflector, RedStone and XOXNO oracle contracts, and Aquarius pools, are read during each request.

- **Governance:** the owner. Source changes go through timelocked proposals. A holder of governance's ORACLE role can narrow a sanity band immediately; widening one requires a new configuration.

Usage by participant

- **Governance** configures each listed asset's sources before the market goes live, and adjusts them over time.
- **Anyone**, including integrators and monitoring tools, can read prices and statuses.

Documentation and code

- The endpoint reference says `set_oracle` keeps the stored token decimals on replacement. The code rejects a replacement whose decimals differ.
- The tolerance has a lower and an upper ratio. Both are validated when set, but only the upper one is used when prices are compared.
- The Reflector "TWAP" mode is a plain average of the returned records, not a time-weighted one. The client's documentation says so; only the name differs.

Price Aggregator — Invariants

INV-PRICE-01 — Unsafe prices are never returned. `prices` never returns a price that is stale, zero or negative, outside the asset's sanity band, or built from sources that disagree beyond their tolerance; it reverts instead. *Source: client documentation.*

INV-PRICE-02 — Two sources need both. For an asset with two sources, a price is returned only when both are usable and agree within the tolerance, and it lies between the two. One source alone is never used as a fallback. *Source: client documentation.*

INV-PRICE-03 — Prices are fresh. Every returned price, and every source it was built from, is within the asset's staleness limit. Prices dated more than a minute in the future are rejected. *Source: client documentation.*

INV-PRICE-04 — Market prices are smoothed or cross-checked. No configuration is accepted whose price depends only on raw spot market readings without an averaged or fundamental second source.

INV-PRICE-05 — LP prices cannot be pushed up by trading. Trading in an Aquarius pool cannot raise the price of its LP token above the value, at oracle prices, of the tokens one share can be redeemed for.

INV-PRICE-06 — Price resolution always terminates. Prices built from other prices nest at most three levels and never in a cycle.

INV-PRICE-07 — Configuration is controlled and consistent. Only the owner, governance, can set sources, tolerances and sanity bands or upgrade, and replacing an asset's sources never changes the token decimals its prices are scaled to. A sanity band can only be narrowed in place; widening it requires a full reconfiguration through the delay. *Source: client documentation.*

Swap Aggregator

The swap aggregator (`contracts/swap-aggregator/`) is a DEX router. It executes a swap route across Soroswap, Aquarius, Phoenix, Sushi and CometDex, takes optional fees, and pays out the result if it meets the route's minimum. The controller uses it inside its strategies, and anyone can call it directly. It has its own admin, separate from lending governance.

Swap Aggregator — Description

How it works

The caller signs a swap request with an input amount and an encoded route. The route lists the tokens involved, the hops (each a swap on a venue, or an Aquarius liquidity add or remove), how much each hop uses, the minimum output and an optional referral.

The router pulls the input from the caller, runs the hops, takes fees, checks the minimum output, and pays the caller. It treats venues as untrusted: it credits each hop only with the balance change it can measure. Small leftovers after the route go to the admin's fee balance; larger ones make the swap fail.

Fees apply only when the route names an active referral. The total is a static fee plus the referral's fee, capped at 10%. A whitelist decides whether the fee is taken from the input or the output token.

Interface

The entry points are in `contracts/swap-aggregator/src/lib.rs` .

Function	Caller	Purpose
`execute_strategy`	anyone (signs as sender)	Run a swap route and receive the output
`claim_referral_fees`	anyone	Pay a referral's fees to its owner
`set_static_fee` , `add_to_whitelist` , `remove_from_whitelist`	admin	Fee settings
`add_referral` , `set_referral_fee` , `set_referral_active` , `set_referral_owner`	admin	Manage referrals
`claim_admin_fees` , `sweep_balance`	admin	Withdraw collected fees or unreserved balances
`upgrade` , `transfer_ownership` , `accept_ownership`	admin	Code and ownership

Views report fees, referrals, the whitelist and fee balances.

Interactions

- **Controller:** calls `execute_strategy` during `multiply` , `swap_debt` , `swap_collateral` and `repay_debt_with_collateral` , with a route supplied by the user. The controller authorizes one transfer of the input, measures the output itself, and refunds unused input. It relies on the router for the route's minimum output.
- **DEX venues:** called once per hop.

- **Lending governance:** chooses which router the controller uses, but cannot administer or upgrade the router.

Usage by participant

- **Users and integrators** call it directly to swap their own tokens.
- **Leverage users** reach it through the controller's strategies, supplying the route.
- **The admin** sets fees and referrals and collects fees.
- **Referral owners** receive a share of fees on routes that name their referral.

Documentation and code

- The README says the fee cap bounds each fee and their sum. The admin functions check each fee separately; the sum is checked only during a swap, so a configuration whose sum exceeds the cap is accepted and then makes every swap with that referral fail.
- Referral ids are 64-bit, but a route can name only a 32-bit id.

Swap Aggregator — Invariants

The client's invariants cover the router mainly as the controller uses it.

INV-SWAP-01 — Only the sender's tokens are used. A swap pulls exactly the input amount from the signing sender and from no one else.

INV-SWAP-02 — The sender gets at least the minimum. A swap either sends at least the route's minimum output, after fees, to the signing sender, or reverts.

INV-SWAP-03 — Each hop is credited with what it delivered. A hop is credited only with the output the venue actually delivered, and must spend exactly the input assigned to it.

INV-SWAP-04 — The router's own funds stay with their owners. Collected admin and referral fees, and any other tokens the router holds, are never paid out to a swap sender. Admin fees can be withdrawn only by the admin, and referral fees only to the referral's owner.

INV-SWAP-05 — Fees are bounded. The total fee on a swap never exceeds 10%, and no fee is charged unless the route names an active referral.

INV-SWAP-06 — Malformed routes are rejected. A route that refers to tokens or amounts outside its own lists, or is otherwise malformed, reverts.

INV-SWAP-07 — Swaps leave at most dust behind. After a swap, the router keeps at most a small residual of any token involved; anything larger makes the swap revert. *Source: client documentation.*

INV-SWAP-08 — Only the admin configures the router. Fees, referrals, the whitelist, sweeps and upgrades require the router's admin.

Position NFT and XOXNO Oracle

Two smaller components. The Position NFT represents lending accounts, and is controlled by the controller. The XOXNO oracle is a price feed run by XOXNO's own signers, which the price aggregator can use as a source.

Position NFT

The position NFT (`contracts/position-nft/`) is a standard non-fungible token built on OpenZeppelin's Stellar library. Each token is one lending account: the token id is the account id, and whoever holds the token controls the account.

How it works

The controller mints a token when an account is created and burns it when the account closes. Ids are sequential; id 0 is reserved, so the first account is 1. Holders can transfer and approve tokens like any NFT. Transferring a token hands over the whole account, collateral and debt included. The delegates the previous holder had added stop working, but are not deleted: if the token returns to that holder, they work again, unless an owner in between changed the delegate list (I-01).

Interface

Function	Caller	Purpose
`mint` , `burn` , `upgrade`	controller only	Create and remove account tokens; replace the code
`renew`	anyone	Extend a token's storage lifetime
`transfer` , `transfer_from` , `approve` , `approve_for_all`	holder or approved address	Standard NFT transfers and approvals
`owner_of` , `balance` , `token_uri` , `total_supply` , ...	anyone (views)	Standard NFT and enumeration views

Interactions

The controller deploys the NFT with itself as its authority, and reads `owner_of` whenever it checks who controls an account.

Usage by participant

Account holders keep, transfer or sell their account by moving the token. Marketplaces and wallets can treat it as any NFT. Keepers may renew storage.

Documentation and code

- The contract's README names OpenZeppelin revision `fbfde388` ; the workspace pins `59b98f8` . Both are library version 0.7.1.

XOXNO Oracle

The XOXNO oracle (`contracts/xoxno-oracle/`) is a price feed run by XOXNO. Registered signers submit prices, and the contract stores a price only when enough of them agree. It has its own admin, separate from lending governance.

How it works

Each signer submits its latest price for a feed. On every submission the contract recomputes the feed's price from the signers' recent submissions:

1. Submissions that are too old are ignored.
2. The remaining ones must be close together in time.
3. At least `threshold` of them must remain; small groups must also agree on price within a set spread.
4. The stored price is the lower median of the group.

If too few signers agree, the previous price is kept until it expires. Prices are in USD with 8 decimals. The contract can be read in RedStone's format, which is how the price aggregator reads it, and in Reflector's format.

Most configured feeds are tokenized fund shares, such as Spiko funds. In the repository's mainnet configuration the Spiko feeds are marked disabled, while an `AQUA` feed is not, and the mainnet AQUA market lists it as its second price source (`configs/mainnet/markets.json`).

Interface

Function	Caller	Purpose
`submit_price` , `submit_prices`	registered signer	Submit prices for one or more feeds
`add_signer` , `remove_signer` , `set_threshold`	admin	Manage the signer set
`set_max_stale_seconds` , `set_max_submission_age_seconds` , `set_max_relative_skew_seconds` , `set_max_cluster_spread_bps`	admin	Freshness and agreement limits
`register_feed` , `add_feed` , `remove_feed` , `purge_feed` , `recompute_feeds` , `set_resolution`	admin	Manage feeds
`upgrade` , `transfer_ownership` , `accept_ownership`	admin	Code and ownership
`read_price_data` , `lastprice` , `prices` , ...	anyone (views)	Read prices in RedStone or Reflector format

Interactions

The price aggregator reads it as one possible source; governance decides which assets use it. The oracle does not know who reads it.

Usage by participant

- **Signers** are off-chain bots that submit prices regularly. Where they take their prices from is not described in the repository.
- **The admin** manages signers, thresholds and feeds.
- **Anyone** can read prices.

Documentation and code

- The README says the stored price is "a median". The code stores the lower median; the client's threat model says so correctly.

Position NFT and XOXNO Oracle — Invariants

The position NFT can also break global INV-SYS-02; the XOXNO oracle can break INV-SYS-08 through the prices it serves.

Position NFT

INV-NFT-01 — Only the controller creates, removes or upgrades. Minting, burning and upgrading succeed only when called by the controller.

INV-NFT-02 — One token per account, never reused. Each token id is the id of exactly one lending account, ids are issued in order starting at 1, and a burned id is never issued again. *Source: client documentation.*

INV-NFT-03 — Only the holder or an approved address moves a token. A token changes hands only through a transfer signed by its holder or by an address the holder approved. Only the controller can burn it.

INV-NFT-04 — A transfer hands over full control. Once a token is transferred, neither the previous holder nor the delegates they added can act on the lending account, unless the token returns to that holder. *Source: client documentation.*

XOXNO oracle

INV-XO-01 — Only registered signers submit prices. A price submission succeeds only if signed by an address in the current signer set.

INV-XO-02 — A stored price needs a quorum. A new price is stored only when at least `threshold` different signers have submitted fresh, close-in-time prices for that feed.

INV-XO-03 — A minority cannot set the price. Colluding signers fewer than the threshold cannot move the stored price further from the honest signers' prices than the configured spread allows.

INV-XO-04 — One signer cannot freeze the price. When enough honest signers submit agreeing, fresh prices, the stored price updates, whatever a single other signer submits.

INV-XO-05 — Reads are fresh. A read never returns a price older than the configured staleness limit, and submissions dated more than a minute in the future are rejected.

INV-XO-06 — Signers cannot go back in time. A signer cannot replace its submission for a feed with one that is older.

INV-XO-07 — Only the admin configures the oracle. Signers, threshold, freshness limits, feeds and upgrades require the oracle's admin, and the threshold always stays between 1 and the number of signers.

Governance

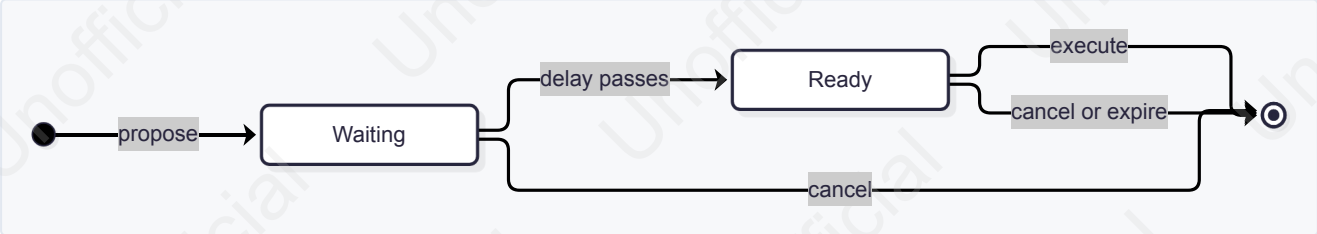
Governance (`contracts/governance/`) owns the controller and the price aggregator. Every change to them is a typed proposal that waits out a delay before anyone can execute it; a few emergency actions take effect immediately. It is built on OpenZeppelin's Stellar timelock and access-control library.

Governance roles

Role	What it can do
Owner	Everything below, plus the changes reserved to the owner: upgrades, price sources, the swap aggregator, the revenue address, Blend approvals, role grants, ownership transfers and the delay itself. Always holds every role.
PROPOSER	Propose any other change: listings, risk parameters, hubs, spokes, markets, unpausing, position managers
EXECUTOR	Nothing in practice: anyone can execute a ready proposal without naming an executor (I-02)
CANCELLER	Cancel proposals before they run
GUARDIAN	Immediately pause the controller, tighten a listing's flags, or create hubs and spokes
ORACLE	Immediately narrow a price's sanity band

At deployment the deployer receives every governance role. The code keeps some apart: an account other than the owner cannot be both EXECUTOR and CANCELLER. Since EXECUTOR restricts nothing, neither does this separation (I-02).

How a proposal works



Each kind of change has a delay tier: **Standard** waits the configured minimum delay; **Sensitive** waits at least that and at least 12 ledgers; **Recovery** waits at least 518,400 ledgers (about 30 days). A ready proposal can be executed for about 7 days (120,960 ledgers); after that it expires. Recovery proposals, used only to replace the set of cancellers, cannot be cancelled.

Interface

Function	Caller	Purpose
`propose`	PROPOSER (owner for owner-only changes)	Schedule a change
`execute`, `execute_self`	anyone	Run a ready change on the controller or aggregator, or on governance itself
`cancel`	CANCELLER	Cancel a pending change
`pause`, `set_spoke_asset_flags`, `create_hub`, `add_spoke`	GUARDIAN	Immediate emergency and setup actions
`set_sanity_band`	ORACLE	Immediately narrow a price range
`deploy_controller`, `deploy_price_aggregator`	owner	One-time deployment
`revoke_role_immediate`	owner	Remove a GUARDIAN or ORACLE immediately
`propose_canceller_reset`, `execute_canceller_reset`	owner / anyone	Replace the cancellers through the Recovery tier
`accept_ownership`	new owner	Complete an ownership transfer

Views report governance roles, the delay, and each proposal's state.

Usage by participant

- **The owner** deploys the core contracts, grants governance roles, and proposes upgrades and price-source changes.
- **Proposers** carry out routine configuration through proposals.
- **Cancellers** watch pending proposals and stop bad ones within the delay.
- **Guardians and oracle operators** respond to emergencies immediately.
- **Anyone** can execute a proposal once it is ready.

Documentation and code

- The client's threat model lists listing, cap, curve and limit changes as what a proposer other than the owner can schedule. The code lets such a proposer schedule every change not on the owner-only list (`timelock/lifecycle.rs`), including registering position managers, forcing bad-debt cleanup, removing spokes and unpausing.

Governance — Invariants

Governance can also break global INV-SYS-01 and INV-SYS-03.

INV-GOV-01 — Changes wait for the delay. After deployment, every change governance makes to the controller or the price aggregator runs only after its delay has passed, except the immediate actions covered by INV-SYS-03.

INV-GOV-02 — The owner's powers stay with the owner. The changes reserved to the owner (the governance-owner rows of Permissions at a glance) can be proposed only by the owner, and no other role can obtain these powers.

INV-GOV-03 — Other roles cannot grant power over user funds. A role holder who is not the owner cannot schedule a change that gives anyone new authority to move users' funds, such as enabling a position manager that users once delegated to.

INV-GOV-04 — The delay never shrinks. The minimum delay can only stay the same or grow, and it is never zero. *Source: client documentation.*

INV-GOV-05 — Each proposal runs at most once, inside its window. A proposal runs only after its delay and before it expires, and a cancelled proposal never runs.

INV-GOV-06 — Cancellers can stop changes, with two exceptions. A canceller can stop any pending proposal except recovery proposals, and no role holder can cancel the proposal that revokes its own role.

INV-GOV-07 — Roles stay separated. No address other than the owner holds both EXECUTOR and CANCELLER, and the last PROPOSER cannot be removed.

INV-GOV-08 — A new owner has full control. After an ownership transfer the previous owner holds no role, and nothing it scheduled before the transfer can give it a role back.

DeFindex Strategy

The DeFindex strategy (`contracts/defindex-strategy/`) lets a DeFindex vault earn lending interest in one market. For each vault it opens and owns one controller account that only supplies. Its market, spoke and controller are fixed at deployment; it has no admin and cannot be upgraded.

Function	Caller	Purpose
`deposit`	vault	Supply tokens to the vault's account
`withdraw`	vault	Withdraw to a chosen address; withdrawing everything closes the account
`balance`	anyone	The vault's current supplied amount
`harvest`	vault	Report the current share price; moves no funds
`asset`	anyone	The configured token

Vaults use it like any DeFindex strategy; the strategy calls the controller's `supply` and `withdraw` on their behalf.

DeFindex Strategy — Invariants

INV-DFX-01 — Only the vault acts for itself. Deposits, withdrawals and harvests for a vault succeed only when signed by that vault.

INV-DFX-02 — Vaults are isolated. A vault's deposits go only to its own lending account, and a vault can withdraw only up to its own balance.

INV-DFX-03 — The reported balance is the real balance. A vault's reported balance equals what its lending account has supplied, including interest.

INV-DFX-04 — The strategy never borrows. The strategy's lending accounts only supply; they never carry debt and have no delegates.

INV-DFX-05 — A full exit closes the account. Withdrawing a vault's whole balance closes its lending account and clears the vault's record, so a later deposit opens a new account.

INV-DFX-06 — Configuration is fixed. The market, spoke and controller cannot change after deployment, and the strategy cannot be upgraded.

Findings

Every finding from every workstream, in one place. The chapter that produced a finding names it and links here; the finding itself is written once.

Security findings (`S-…`) have a security consequence. Each carries a severity, a difficulty, a recommended action and a status; see Finding ratings.

Informative findings (`I-…`) have no security impact and carry no rating.

Candidates raised by the Specula runs, recorded in Appendix A, are under joint triage with XOXNO; those we confirm are added here as findings.

ID	Finding	Area	Found by	Severity	Difficulty	Recommended action	Status
S-01	A withdraw amount of zero closes the market and can leave an account at its borrowing limit	Controller	Code review	Low	High	Fix Design	Not Addressed
S-02	LTV-only restamp on a loosened listing leaves a position's LTV and LT inconsistent, exposing borrowers to unexpected liquidation	Controller	LLM-assisted code review	Low	Medium	Fix Code	Not Addressed
S-03	A borrowable market with a 100% utilization cap can be frozen until the pool is upgraded	Pool	LLM-assisted code review	Medium	High	Fix Code	Not Addressed
S-04	A market whose supply value passes about 1.7e11 whole units freezes on the next accrual	Pool, Controller	Code review	Low	High	Fix Code	Not Addressed
I-01	Delegate grants survive an NFT transfer and reactivate when it returns	Controller	Code review	—	—	Document Prominently	Acknowledged
I-02	The optional `executor` argument and the EXECUTOR role restrict nothing	Governance	Code review	—	—	Fix Code	Not Addressed
I-03	Endpoints take key-value vectors where a map would rule out duplicate keys	Controller	Code review	—	—	Fix Code	Not Addressed
I-04	Proposal salts separate pending operations; they do not prevent re-proposal	Governance	Code review	—	—	Document Prominently	Not Addressed
I-05	Governance role-change events name the owner as the caller	Governance	Code review	—	—	Fix Code	Not Addressed
I-06	Blend pools and position managers can only be revoked through the timelock	Governance, Controller	Code review	—	—	Fix Design	Not Addressed
I-07	In-band sentinel values give ordinary numbers a second meaning	Controller, Governance, Swap aggregator	Code review	—	—	Fix Design	Not Addressed

ID	Finding	Area	Found by	Severity	Difficulty	Recommended action	Status
I-08	The rate-model validation lets the cap flatten the curve below optimal utilization	Pool	Code review	—	—	Fix Code	Not Addressed

The same table, without the recommended action, opens the Executive Summary.

Security findings

[S-01] A withdraw amount of zero closes the market and can leave an account at its borrowing limit

Severity: Low | Difficulty: High | Recommended Action: Fix Design

Description

`withdraw` treats an amount of `0` as a request to close the position. The controller rewrites the zero to `i128::MAX`, its full-close sentinel (`positions/supply.rs:190-194`), and the pool withdraws the whole position for that value, as it does for any request at or above the position's value (`common/src/rates/scaling.rs:105-123`). When a payload names the same market twice, the zero leg wins over the positive one (`payments.rs:117`). The client's endpoint reference documents this behaviour.

After a withdrawal the controller checks that LTV-weighted collateral still covers the debt, that the health factor is at least 1, and that the collateral floors hold (`risk/validation.rs:29-61`). An unintended full close succeeds whenever the remaining collateral covers the debt, and it can leave the account exactly at its borrowing limit, the state a maximum borrow produces. A price drop larger than the gap between the collateral's LTV and its liquidation threshold then makes the account liquidatable (`views.rs:47-48`), and the liquidator takes a bonus from the user's collateral.

Other entry points and views give a numeric value a second meaning in the same way. None of those cases leads to a loss; I-07 lists them.

Scenario

1. An account holds collateral in markets A and B and has debt.
2. A client sends `withdraw` with the leg `(A, 0)`. A wallet may submit an empty amount field as zero, or a position-management bot may compute a zero adjustment and send it instead of skipping the leg. The owner, or the bot acting as delegate, signs a transaction that reads as a withdrawal of nothing.
3. The controller closes A in full. The risk gate passes because B still covers the debt at LTV, and the account ends at or near its limit.
4. B's price falls by more than its LTV-to-threshold gap. A liquidator repays part of the debt and takes the bonus from the account's collateral.

Difficulty is High. The case needs a client or bot that produces the zero, an account whose remaining collateral sits near the limit, and an adverse price move before anyone reacts. An attacker gains little, because the bonus

goes to the first liquidator to arrive; the realistic trigger is a defect. Severity is Low: the liquidation bonus and fees bound the loss, on an account the protocol already allowed to sit at that limit.

Recommendations

Reject `0` in withdraw legs and keep `i128::MAX` as the only close instruction, or replace the amount with an `Exact(i128) | All` enum so that no value that reads as nothing can close a position. Reject payloads that name the same market with both a close and a positive amount.

Status

[S-02] LTV-only restamp on a loosened listing leaves a position's LTV and LT inconsistent, exposing borrowers to unexpected liquidation

Severity: Low | Difficulty: Medium | Recommended Action: Fix Code

Description

Every supply position keeps its own copy of the listing's risk terms: loan-to-value (LTV), liquidation threshold (LT), liquidation bonus and liquidation fee. An edit to a listing does not rewrite these copies. User operations refresh them lazily, and not all in the same way:

- **All four terms**, on the supply position an operation moves (`refresh_supply_risk_params`):
  - `supply` , and the deposits of the account strategies;
  - `withdraw` when a balance remains, and the withdrawals of the account strategies;
  - `update_account_threshold(has_risks = true)` , on every listed supply position.

  LT, bonus and fee are skipped only when the change favours liquidators, the account has debt, and its health factor (HF) under the new LT would fall below 1.05. A loosening is never skipped.
- **The LTV only**, on every other listed supply position (`restamp_listed_supply_ltv`):
  - `borrow` , `withdraw` and every account strategy, just before their solvency checks;
  - the `get_ltv_collateral_usd` view, in memory;
  - `update_account_threshold(has_risks = false)` , through `refresh_supply_risk_params` with `RiskRefreshScope::LtvOnly` .

  For example, when Alice borrows USDC against WBTC, only her WBTC position's LTV is updated.

`repay` and liquidation refresh nothing.

The borrow limit weights each leg by its stored `min(LTV, LT)` (`risk/totals.rs:95` , `:190`), and the health factor uses the stored LT (`risk/totals.rs:196`). The gap between the two is the margin a borrower has before liquidation. A listing sets it as LT − LTV, and listing validation keeps it positive by requiring LT > LTV (`common/src/validation.rs:95-107`).

When governance loosens a listing by raising both its LTV and its LT, the LTV-only restamp writes the new LTV next to the old LT. The margin of an existing position becomes old LT − new LTV, short of the listing's new LT −

new LTV by the LT increase. If the new LTV reaches the old LT, the margin is zero: the stored copy has LTV ≥ LT, a combination listing validation forbids, and the borrow limit equals the liquidation line. A borrower who borrows the limit the controller reports ends at an HF of exactly 1 and becomes liquidatable as soon as interest accrues.

The edit alone makes no account liquidatable, because the HF still uses the old LT. The harm comes from the next borrow, withdrawal or strategy up to the reported limit.

The client's listing runbook (`docs/reference/runbooks/liqvid-listing-params.md` , section 10) documents the LTV-only refresh, and discusses it for an edit that lowers the LT. It does not cover an edit that raises the LTV.

How it was found

LLM-assisted code review of the controller's risk-parameter refresh paths. We confirmed it with a test written for the repository's test harness and run at the audited commit. The scenario below uses its values, and ends with the test.

Scenario

WBTC ($60,000) is listed as collateral at LTV 75%, LT 80%, bonus 5%. The debt asset is USDC ($1), with 100,000 USDC of liquidity supplied by Bob.

1. Alice supplies 0.5 WBTC ($30,000). Her stored WBTC terms are LTV 7500, LT 8000. Her borrow limit is $22,500.
2. Governance edits WBTC's listing to LTV 80%, LT 85% (`EditAssetInSpoke`). Alice's stored terms are unchanged. `get_ltv_collateral_usd` now reports $24,000: min(8000, 8000) of $30,000.
3. Alice borrows 24,000 USDC, the reported limit. The borrow restamps her stored terms to LTV 8000, LT 8000. Her HF is exactly 1.0: $30,000 × 80% / $24,000.
4. One ledger later, interest has taken her HF below 1 and the account is liquidatable. A liquidation takes the bonus from her collateral.

Had her copy received the full listed terms, LTV 8000 and LT 8500, as a supply of even one raw unit of WBTC before step 3 would do, the same borrow would leave an HF of 1.0625: $30,000 × 85% / $24,000. The listing's 5% margin would be intact.

The test passes at the audited commit. It uses only the harness's existing helpers, and runs in the controller suite (`tests/test-harness/tests/controller/`), for example appended to `ltv_restamp_events.rs` with `use controller::constants::WAD;` added to its imports:

```
/// WBTC relisted from 7_500 / 8_000 to 8_000 / 8_500. The borrow restamps the
/// stored LTV to 8_000 but keeps the stored threshold at 8_000, so a borrow at
/// the reported limit leaves no margin before liquidation.
#[test]
fn a_raised_ltv_restamp_keeps_the_old_threshold_and_removes_the_margin() {
    let mut t = LendingTest::new().three_asset_usdc_eth_wbtc().build();
    t.supply(BOB, "USDC", 100_000.0);
    t.supply(ALICE, "WBTC", 0.5);
    let account = t.account_id(ALICE);
    t.edit_asset_in_spoke("WBTC", HARNESS_SPOKE, true, true, 8_000, 8_500, 500);

    let limit = t.ctrl_client().get_ltv_collateral_usd(&account);
    assert_eq!(limit, 24_000 * WAD);
    t.borrow(ALICE, "USDC", 24_000.0);
```

```
    let (supplies, _) = t.ctrl_client().get_account_positions(&account);
    let wbtc = supplies.get(hub_asset(t.resolve_asset("WBTC"))).unwrap();
    assert_eq!(
        (wbtc.loan_to_value, wbtc.liquidation_threshold),
        (8_000, 8_000)
    );
    assert_eq!(t.health_factor_raw(ALICE), WAD);

    t.advance_time(5);
    assert!(t.can_be_liquidated(ALICE));
}
```

Preconditions:

- Governance raises a listing's LTV and LT. The stored margin falls short of the listing's by the LT increase, and is zero if the new LTV reaches the old LT. A PROPOSER can schedule the edit. Mainnet gaps are 300 to 1000 basis points (`configs/mainnet/spokes.json`).
- The position predates the edit, and has not since been supplied to, withdrawn from, or refreshed with `update_account_threshold(has_risks = true)` . No keeper or operations script calls that.
- The borrower borrows, withdraws or runs a strategy up to the reported limit.

Only the borrower loses: the liquidation bonus on the seized collateral. No third party can trigger it.

Recommendations

Apply the listing's LT, bonus and fee whenever the LTV is restamped and the change does not favour liquidators: in `restamp_listed_supply_ltv` , reuse the gating in `apply_gated_liquidation_params` . A loosening then reaches the borrow, withdrawal and strategy paths in full, as it already does on supply.

Where the full tuple cannot be applied, because the change favours liquidators and the 1.05 gate fails, never store an LTV at or above the stored LT. Clamp the restamped LTV to keep at least the listing's LT − LTV gap below the stored LT.

Compute `get_ltv_collateral_usd` from the same refreshed terms the next borrow would use, so that the reported limit keeps the listing's margin.

Add the scenario above to the test suite, inverted: after the fix, borrowing the reported limit must leave the HF above 1.

Status

[S-03] A borrowable market with a 100% utilization cap can be frozen until the pool is upgraded

Severity: Medium | Difficulty: High | Recommended Action: Fix Code

Description

Every pool operation on a market first accrues interest, and every accrual computes the market's utilization (`contracts/pool/src/interest.rs:20-33` , `common/src/rates/simulate.rs:62`). Utilization is debt value

divided by supply value (`common/src/rates/curve.rs:100-106`). The division is a fixed-point one that panics with `MathOverflow` (#33) when the result does not fit in an `i128` (`common/src/math/fp.rs:55-57` , `fp_core.rs:108-118`). In RAY units that happens once debt exceeds supply by a factor of about $1.7 \cdot 10^{11}$.

Once a market's ratio passes that bound, every operation on it reverts: supply, withdraw, borrow, repay, liquidation, bad-debt cleanup, revenue claims and `update_indexes` . So does the rate-model update, which accrues before it replaces the parameters (`contracts/pool/src/ops/market.rs:52-53`). Nothing inside the market can bring the ratio back down. Only a pool code upgrade, which does not accrue (`contracts/pool/src/lib.rs:122-126`), recovers it.

Normally the utilization guard keeps debt below supply. It is skipped when the market's `max_utilization` is 100% (`contracts/pool/src/guards.rs:19-35`). The only other borrow guard, the liquidation buffer, reserves 2% of the supply value rounded down (`guards.rs:39-49`), which is zero when the supply is worth less than one token unit. At a 100% cap, an account can therefore borrow a market's last cash, using collateral in another market, while its supply is that small, and the ratio jumps far past the bound.

In summary, a market locks when:

- it is borrowable on a spoke and its `max_utilization` is 100%;
- its only supply is dust worth less than one token unit, such as leftover protocol revenue after every supplier has withdrawn;
- someone borrows the market's remaining cash, using collateral in another market, while that dust is the market's only supply.

In any later ledger, the first operation on the market accrues interest, the utilization division overflows, and from then on every operation on the market reverts until the pool is upgraded.

Parameter validation accepts a 100% cap (`common/src/types/pool.rs:212`), and nothing ties the cap to whether a spoke lists the asset as borrowable. Today 24 mainnet markets and 2 testnet markets have a 100% cap; none of them is borrowable on any spoke, and every borrowable market has a cap of 90–95% (`configs/{mainnet,testnet}/markets.json` , `spokes.json`). A governance edit that makes one of them borrowable, with a positive rate, without lowering its cap, exposes it.

How it was found

LLM-assisted code review of the pool's rate arithmetic. We confirmed it with a test written for the repository's test harness and run at the audited commit. The scenario below uses its values, and ends with the test.

Scenario

USDC ($1, 7 decimals) is borrowable with a 100% utilization cap, a rate curve with a base rate of 0, and a positive reserve factor. ETH ($2,000) is collateral.

1. Alice supplies 1,000 USDC. Bob supplies 1 ETH and borrows one raw unit of USDC.
2. A ledger later, Bob repays in full. The interest on one unit leaves a sub-unit amount of protocol revenue in the market's supply, and one unit of extra cash.
3. Alice withdraws all her USDC. The revenue dust is now the market's only supply, and the market holds one raw unit of cash.
4. Bob borrows that unit. The utilization guard is skipped at a 100% cap, and the liquidation buffer is zero.
5. A ledger later, `update_indexes` on USDC and Bob's repayment both revert with `MathOverflow` . So does every other operation on the market, until the pool is upgraded.

With a 95% cap, the borrow in step 4 is refused and the market keeps working. The refusal itself is a `MathOverflow`, not `UtilizationAboveMax` (#127): the guard's own division overflows on the same ratio before it compares.

One actor can play every role: supplying, borrowing one unit from a second account, repaying, withdrawing, and borrowing the remaining cash. It needs only that nobody else supplies the market, as when a market has just become borrowable, and about $7 of collateral, enough for the $5 minimum borrow floor after LTV. The test uses a base rate of 0 so that one cycle leaves dust small enough; the cash borrowed must be about $1.7 \cdot 10^{11}$ times the dust's value.

The test passes at the audited commit. It uses only the harness's existing helpers, and runs in the controller suite (`tests/test-harness/tests/controller/`), for example appended to `max_utilization.rs` with `eth_preset` and `MarketPreset` added to its `test_harness` imports:

```
/// At a 100% cap the utilization guard is skipped, so the last unit of cash can
/// be borrowed while the market's only supply is sub-unit revenue dust. The next
/// accrual overflows `borrowed / supplied`, and every operation on the market
/// reverts.
#[test]
fn borrowing_the_last_cash_while_supply_is_dust_at_a_full_cap_locks_the_market() {
    let mut t = LendingTest::new()
        .with_market(MarketPreset {
            initial_liquidity: 0.0,
            ..usdc_preset()
        })
        .with_market(eth_preset())
        .with_market_params("USDC", |p| {
            p.max_utilization = RAY;
            p.base_borrow_rate = 0;
        })
        .build();

    // A one-unit loan repaid a ledger later leaves sub-unit revenue as the
    // only supply once ALICE leaves, and one unit of cash.
    t.supply(ALICE, "USDC", 1_000.0);
    t.supply(BOB, "ETH", 1.0);
    t.borrow_raw(BOB, "USDC", 1);
    t.advance_time(5);
    t.repay_raw(BOB, "USDC", 10);
    t.withdraw_all(ALICE, "USDC");

    t.borrow_raw(BOB, "USDC", 1);
    t.advance_time(5);
    assert_contract_error(t.try_update_indexes_for(&["USDC"]), errors::MATH_OVERFLOW);
    assert_contract_error(t.try_repay(BOB, "USDC", 1.0), errors::MATH_OVERFLOW);
}
```

Preconditions:

- Governance makes a market with a 100% cap borrowable on a spoke, with a positive rate curve and reserve factor. No network has such a market today.
- Nobody else supplies the market when the attack runs, as at listing time.

In the minimal attack only protocol dust and the attacker's own position are frozen. If other accounts already hold positions in the market, for example a first supplier front-run at listing, those positions are frozen too, and cannot be liquidated, until the pool is upgraded.

Recommendations

Make utilization saturate instead of overflowing: in `utilization`, return the curve's maximum point when debt is at or above supply, rather than dividing. Rates on healthy markets do not change, and no ratio can lock a market.

Keep debt at or below supply at a 100% cap: in `require_utilization_below_max`, do not return early when `max_utilization` is 100%, and make its own comparison overflow-safe.

Refuse to list an asset as borrowable while its pool market has a 100% cap, or cap `max_utilization` below 100% in parameter validation.

Add the scenario above to the test suite, inverted: after the fix, the market must keep working after step 5.

Status

[S-04] A market whose supply value passes about 1.7e11 whole units freezes on the next accrual

Severity: Low | Difficulty: High | Recommended Action: Fix Code

Description

The pool keeps every amount at 27 decimals. `Ray::from_asset` multiplies a token amount by `10^(27 - decimals)` and panics with `MathOverflow` when the product leaves `i128` (`fp.rs:140-147`, `fp_core.rs:218-223`). Because the scale is fixed, the largest representable amount is `i128::MAX / 10^27`, about 170,141,183,460 whole units, for every decimals setting from 0 to 18.

The same ceiling binds the market total. Accrual values all supply as `supplied * supply_index` at RAY precision (`scaling.rs:14-16`, called from `simulate.rs:60-61`), and every mutating entry point accrues before it does anything else (`ops/mod.rs:30-47`, `interest.rs:20-33`). Once the value of a market's supply or debt crosses the ceiling, whether through a deposit or through interest, the next accrual overflows and every leg on the market reverts: withdrawals, repayments, liquidation withdrawals and seizures alike. `update_params` and `update_indexes` accrue too (`ops/market.rs:52-73`), so the only way out is `upgrade`. Other markets are unaffected: the cache is loaded per market.

The same accrual step holds a second overflow, one line later: the utilization division of S-03 fails once debt exceeds supply by the same factor, which a 100% utilization cap makes reachable from a dust supply. The two are independent. Here the market's value overflows before utilization is computed, so a saturating utilization would not prevent it, and the ceiling recommended below would not prevent S-03, whose market values stay tiny.

XOXNO documents the limit. The formulas reference lists the token-to-RAY maximum and states that "value overflow can occur before the index ceiling and block repayment/withdrawal because those operations accrue first", and that "no dedicated ceiling alarm is emitted" (`formulas.md:447-455`); the threat model asks that "caps must account for plausible index growth as well as token balances" (`threat-model.md:317-323`); INV-IDX-01

and ADR-0016 carry the same caveat (`invariants.md:235` , `decisions.md:139`). A test in the tree reproduces the freeze and describes it as "no repay, no withdraw, no liquidation" (`large_positions_and_long_horizons.rs:315-321`).

The controller's caps are the only guard, and they do not close the gap:

- `require_cap_within_asset_domain` bounds `supply_cap` and `borrow_cap` to exactly this ceiling (`validation.rs:48-70` , applied at `config/asset.rs:70-71`). A cap at its maximum passes, and the test in the Scenario supplies at it.
- Caps are enforced per spoke (`spoke_usage.rs:144-155`); the pool holds the sum over all spokes.
- Caps gate entries only. Interest raises the value of positions already inside, and nothing compares that value with the ceiling.
- A bad-debt socialization lowers the supply index, floored at `RAY / 1000` (`interest.rs:73-90` , `constants/pool.rs:9`). `supplied` is also an `i128` , so the share count overflows at a value smaller by the same factor: about 1.7e8 whole units at the floor.

Scenario

Interest, not deposits, is the realistic route, because the ceiling is on value and the index multiplies it. XOXNO's own measurement, recorded in the pull request that added the test (#132), is that a market of one billion whole tokens at a sustained 98% utilization on the XLM curve reaches the ceiling in its fourth year, at an index of about 121. The Certora notes put the largest mainnet supply cap, AQUA at 5e8 tokens, 200 times below the ceiling (`rates_rules.rs:22-36`), which is a margin in index growth, not an impossibility. Tokens whose supply exceeds 1.7e11 units, common among memecoins, reach it by deposits alone.

The shortest route is a cap at the admitted maximum. `max_cap_for_decimals` is the largest cap whose RAY form fits `i128` , so a supply at that cap leaves less than one token of headroom under the ceiling, and the first accrual that adds interest to it overflows in `update_supply_index` (`index.rs:40`); every later call on the market fails there or in `scaled_to_original` . The test below reproduces this at the audited commit with one minute of interest at ten percent utilization. It passes when appended to `tests/test-harness/tests/controller/large_positions_and_long_horizons.rs` , the file that holds XOXNO's own ceiling test, reusing its `big` , `col` and `lift_caps` helpers, after `make build` has produced the pool WASM:

```
cargo test -p test-harness --test controller a_market_supplied_at_the_admitted_cap
```

```
/// The admitted cap puts a market's ray value within one token of `i128::MAX`.
/// Any interest then overflows the next accrual, and every verb accrues first,
/// so a market supplied at its cap freezes on the first ledger after a borrow.
#[test]
fn a_market_supplied_at_the_admitted_cap_freezes_on_the_first_accrual() {
    let mut t = LendingTest::new()
        .with_market(big("BIG7", 7, xlm_curve()))
        .with_market(col())
        .build();
    lift_caps(&t, "BIG7", 7);
    lift_caps(&t, "COL", 7);

    // The largest cap admission accepts for seven decimals: about 1.7e11 whole tokens.
    let cap = max_cap_for_decimals(7);
    t.supply_raw(BOB, "BIG7", cap);
```

```
    t.supply_raw(ALICE, "COL", cap / 5);
    t.borrow_raw(ALICE, "BIG7", cap / 10);

    // Before any interest accrues, the market works.
    t.withdraw_raw(BOB, "BIG7", 1);

    // One minute of interest at ten percent utilization is a few hundred tokens,
    // far more than the headroom left under the ceiling.
    t.advance_time(60);
    assert_contract_error(t.try_update_indexes_for(&["BIG7"]), errors::MATH_OVERFLOW);
    assert_contract_error(t.try_withdraw_raw(BOB, "BIG7", 1), errors::MATH_OVERFLOW);
    assert_contract_error(t.try_repay(ALICE, "BIG7", 1.0), errors::MATH_OVERFLOW);
}
```

With half the cap supplied at the same utilization, the same three calls succeed after the same minute. Only the admitted maximum fails.

Nobody outside governance can cause this: it takes a listing with caps near the ceiling, or a large market left at high utilization for years. Once it happens, no supplier can leave and no position in that market can be liquidated, so a price move during the freeze can create bad debt in other markets until governance upgrades the pool.

Recommendations

Turn the freeze into a refused booking and stop interest at the ceiling. XOXNO drafted exactly that in #196: a `MAX_MARKET_VALUE_RAY` constant one token below `i128::MAX`, an accrual step that clamps index growth so neither total passes it and emits a `MarketValueCeilingEvent` when it does, and a guard on supply entry and fee booking (`constants/pool.rs:39`, `simulate.rs:126-168`, `guards.rs:90-99`). Alongside it, lower `max_cap_for_decimals` by a headroom for interest, check the sum of spoke caps against it, and record the ceiling in the listing runbook so a token's supply is compared against it before listing.

Status

Not addressed. The limit is documented and tested on the client's side, and the fix in #196 was closed unmerged on 23 October 2026 ("the Specula fixes will be redone from scratch in new PRs"). None of the follow-up pull requests #203 to #211 carries it, and `main` at `5aaa81df` has no trace of it. Whether the omission was deliberate is an open question for XOXNO.

Informative findings

I-01 Delegate grants survive an NFT transfer and reactivate when it returns

Recommended Action: Document Prominently | Acknowledged by client

Description

An account owner can name up to 16 delegates, active position managers that may borrow, withdraw and run strategies on the account. The controller keeps one delegate list per account, stamped with the address of the owner who granted it (`storage/account.rs:176-199`). A caller passes the delegate check only if the list's stamp matches the current NFT owner (`account.rs:115-127`).

A transfer of the position NFT does not touch this list; the position NFT contract has no knowledge of delegates. The previous owner's list stays in storage and stops matching. It is not revoked, and it matches again if the NFT returns to that owner.

The grant survives only until an intervening owner changes the list:

- Adding a delegate replaces the stale list (`storage/account.rs:203-221`).
- Removing any delegate, including one on no list, deletes the stale list (`storage/account.rs:236-238`).

The controller has no entrypoint that returns an account's delegates.

This is the documented behavior: INV-AUTH-02 in `docs/reference/invariants.md` and the accounts section of `docs/reference/endpoints.md` both state it. Those are reference documents for developers, though, and the behavior is easy to read the other way: that a transfer hands over the account and leaves nothing of the previous owner's behind.

How it was found

Code review of the controller's account authorization, during the design review.

Scenario

1. Alice owns account 7 and adds position manager M as a delegate.
2. Alice transfers the NFT for account 7 to Bob. M can no longer act on account 7.
3. Bob does not change the delegate list, and transfers the NFT back to Alice.
4. M can again borrow and withdraw from account 7, and route the proceeds to an external recipient. Alice took no action to re-grant it.

The outcome depends on what Bob did in step 3. Had he added or removed any delegate, Alice's grant would be gone. Either way, Alice cannot query which delegates are live.

No one gains authority they were not once given: M must have been granted by Alice, and must still be an active position manager. This is why the finding is informative.

Recommendations

Document the behavior where users and integrators will see it, not only in the reference documentation: in the user-facing documentation, and in any interface that transfers or lists the position NFT. Say that grants come back with the NFT, and that removing each delegate before a transfer is the way to revoke them.

Add a view that returns the delegates currently active for an account, so that an owner or an integrator can check it directly.

Status

XOXNO acknowledged the behavior as intended (lazy update on transfer) and asked that it be documented.

I-02 The optional `executor` argument and the EXECUTOR role restrict nothing

Recommended Action: Fix Code

Description

The three governance entrypoints that run a ready operation take an `executor: Option<Address>` argument: `execute` (`api.rs:57-67`), `execute_self` (`api.rs:177-179`) and `execute_canceller_reset` (`api.rs:195-202`). All three pass it to `authorize_executor` (`timelock/mod.rs:75-82`). With `Some(address)`, that address must authorize the call and hold the EXECUTOR role. With `None`, there is no check at all.

The caller chooses the argument, so any caller can pass `None`. The EXECUTOR role therefore gates nothing: every ready operation can be executed by anyone, whether or not an executor is named, and whether or not anyone holds the role.

Two other pieces of code exist only for the role:

- Granting a role refuses to give one account both EXECUTOR and CANCELLER (`access.rs:139-163`). The separation means nothing when execution is open to everyone.
- The constructor grants EXECUTOR to the admin, and `GrantGovRole` and `RevokeGovRole` can grant and revoke it, through the timelock.

Permissionless execution is the intended design: ADR-0006 and the threat model both say that anyone may execute when no executor is named. The issue is the role and the argument around it:

- They are misleading. They suggest that execution can be restricted, and an operator granting the role, a reader of the role list, or a reader of the tests `non_executor_execute_rejected` and `non_executor_cannot_execute` could take that at face value.
- They are dead code: the argument, the role, the separation check and their tests have to be maintained and reviewed, and have no effect.

How it was found

Code review of the governance contract.

Scenario

1. XOXNO grants EXECUTOR to a dedicated address, intending that only it runs ready operations.
2. An operation becomes ready.
3. Any account calls `execute` with `executor = None` and runs it. The dedicated address was never needed.

Execution is meant to be permissionless, so nothing here is exploitable, and the timelock's guarantees are unaffected. This is why the finding is informative.

Recommendations

Remove the EXECUTOR role and the `executor` argument from `execute`, `execute_self` and `execute_canceller_reset`, and with them the executor/canceller separation check. This makes the code say what it does: execution of a ready operation is permissionless.

Status

We discussed this with XOXNO, who are leaning towards removing the role.

I-03 Endpoints take key-value vectors where a map would rule out duplicate keys

Recommended Action: Fix Code

Description

Several controller entrypoints take a list of key-value pairs, `Vec<(K, i128)>`, where each key should appear once. Each decides for itself what a repeated key means:

Entrypoint	Argument	Key	Repeated key
`supply`	`assets`	`HubAssetKey`	summed
`borrow`	`borrows`	`HubAssetKey`	summed
`withdraw`	`withdrawals`	`HubAssetKey`	summed; a zero overrides
`repay`	`payments`	`HubAssetKey`	summed
`liquidate`, `get_liquidation_estimate`	`debt_payments`	`HubAssetKey`	summed
`flash_position`	`collaterals`	`HubAssetKey`	rejected
`migrate_from_blend`	`debt_caps`	`Address`	rejected

Soroban's `Map<K, V>` cannot hold a repeated key: the host rejects a map argument whose keys are not sorted and unique when it decodes the call (`soroban-env-host` 28.0.2, `host/metered_map.rs:128-135`), before the contract runs. A vector can repeat a key, so the controller has to handle repeats itself. This makes the code more complicated and increases the gas cost of every call.

More complicated code. Each rule is correct and documented in `docs/reference/endpoints.md`, but the aggregation with its separate order vector, the zero-override rule and the duplicate checks exist only because the input type allows repeats. They make the controller harder to understand, maintain and verify: a reader, a new entrypoint and a proof all have to account for which rule applies where.

With a map, "no key appears twice" is an invariant of the type, enforced by the host, and every function that receives one can rely on it. With a vector, it depends on whoever built the vector, so a receiving function either trusts every caller or checks again. `validate_bulk_position_limits` (`risk/validation.rs:65-100`) checks again, although every caller has already made the keys unique, so some inputs are deduplicated twice.

Higher gas cost. Each deduplication inserts every key into a map. Soroban maps are immutable, so each insertion copies the whole map (`soroban-env-host` 28.0.2, `host/metered_map.rs:196-225`), and n insertions cost O(n²). A map argument is checked once, in a single pass, when the host decodes it.

How it was found

Code review of the controller's entrypoints.

Scenario

Every rule behaves as documented, and none puts funds at risk. The cost is in complexity and gas, which is why the finding is informative.

Recommendations

Take `Map<HubAssetKey, i128>` (and `Map<Address, i128>` for `debt_caps`) in these entrypoints. The host then guarantees unique keys, and the aggregation and duplicate checks reduce to validating each amount. Remove the repeated-key handling in `validate_bulk_position_limits` as well; its callers already pass unique keys.

This is an ABI change, and results and events would follow key order rather than input order.

Status

[I-04] Proposal salts separate pending operations; they do not prevent re-proposal

Recommended Action: Document Prominently

Description

A proposal's id is the hash of its target, function, arguments, predecessor and salt (`timelock/mod.rs:182-188`, `api.rs:94-102`). The salt gives two otherwise identical operations different ids while both wait in the queue. It does not outlive execution: `finish_execute` removes the operation's ledger entry and its sidecar state (`timelock/mod.rs:190-197`), and nothing records that the id ran. The same operation, with the same salt, can be proposed and executed again once the delay has passed.

The salt separates pending proposals. Protection against repeating an executed change comes from the delay and the cancellers, as for any new proposal. The governance README describes the salt as the way to give a re-proposed operation a new id and says that execution removes the entry (`README.md:18-24`); it does not say that an executed operation may be scheduled again under its old id.

Recommendations

State the salt's scope in the governance documentation. If an executed operation must never run again under the same id, keep a per-id "done" marker instead of removing the entry.

Status

[I-05] Governance role-change events name the owner as the caller

Recommended Action: Fix Code

Description

Role changes that go through the timelock record the current owner as the caller, whoever proposed or executed them:

- `apply_grant_role` and `apply_revoke_role` pass the owner to the OpenZeppelin `grant_role_no_auth` and `revoke_role_no_auth` helpers, which emit it as the caller (`access.rs:170-174` , `access.rs:207-223`).
- `apply_canceller_reset` passes the owner for every revocation and grant it performs (`access.rs:182-198`).
- `apply_transfer_ownership` emits the owner at execution time as the `from` address, and the mirrored admin-transfer event does the same (`access.rs:125-135` , `access.rs:59-75`).

Anyone may execute a ready proposal, and any PROPOSER may propose `RevokeGovRole` , so the event names an address that did not initiate the change. If ownership changes between proposal and execution, the event names the new owner as the author of the old owner's grant, reset or transfer. The immediate path, `revoke_role_immediate` , names the right address because the owner calls it.

Off-chain monitoring and forensics that read these events attribute role changes to the wrong address. On-chain authorization does not depend on them.

Recommendations

Record the proposer in the operation's sidecar state at proposal time and emit it at execution, or emit a governance event with the operation id next to the OpenZeppelin event so that readers can recover the proposer from the proposal event.

Status

[I-06] Blend pools and position managers can only be revoked through the timelock

Recommended Action: Fix Design

Description

The controller's `revoke_blend_pool` and `set_position_manager` are owner-only (`lib.rs:585-607`), and governance is the owner, so both run only as proposals: `RevokeBlendPool` in the Standard tier and `SetPositionManager` in the Sensitive tier (`op.rs:279-285` , `op.rs:361`). Governance's immediate actions, `pause` , `set_spoke_asset_flags` , `set_sanity_band` , `create_hub` , `add_spoke` and `revoke_role_immediate` (`api.rs:127-172`), do not cover either registry.

A compromised Blend pool or position manager stays usable until a revocation proposal passes the delay: `migrate_from_blend` checks the approval at call time (`migrate_blend.rs:184`), and a delegate acts on every account that granted it while the manager flag is active (`account.rs:115-127`).

The guardian can contain the damage at a cost to everyone. Pausing the controller blocks migrations, borrows and strategies, which carry `#[when_not_paused]` (`lib.rs:93-341`), and pausing the affected listings blocks withdrawals through the exit-leg check (`positions/mod.rs:257-266`). Together they stop every outflow a

compromised manager could trigger, and they halt those paths for every user. The window is the configured delay: 12 ledgers, about one minute, in both deployed configurations (`configs/networks.json:5` , `:43`), and about two days once raised to the value the runbooks anticipate (`liqvid-listing-params.md:125-126`).

Recommendations

Add an immediate, tighten-only revocation for both registries, available to the owner or the guardian like the existing flag ratchet, so that governance can cut off a compromised pool or manager without pausing markets for everyone.

Status

[I-07] In-band sentinel values give ordinary numbers a second meaning

Recommended Action: Fix Design

Description

Several entry points and views treat one value of a numeric argument as a command or a status. Because the sentinel shares the type of the quantity, an unset or mistyped field makes a valid call. The one case with a path to a user loss, a withdraw amount of `0` , has its own rating in S-01; the cases below share the pattern without that consequence.

Cases that can mislead a user or an operator:

- **An account id of `0` creates an account** in `supply` , `multiply` , `flash_position` , `migrate_from_blend` and `SeizeMode::Credit(0)` (`account.rs:95` , `positions/liquidation/mod.rs:148-166`). An unset id opens a second account, bound to the given spoke and possibly leveraged, and leaves the intended account untouched.
- **`get_health_factor` and `is_liquidatable` report a missing account as safe.** Nonexistent, burned and debt-free accounts all return `i128::MAX` and `false` (`views.rs:28-48`). A keeper cannot tell "safe" from "wrong id".
- **Zero has opposite meanings across governance parameters.** `set_min_borrow_collateral_usd(0)` disables the floor (`risk/validation.rs:56`); a `max_utilization` at or above 1 disables the utilization gate (`pool/src/guards.rs:20`); a `supply_cap` or `borrow_cap` of `0` closes the market (`spoke_usage.rs:144-157`). An operator who applies the cap convention to the floor removes a protection without noticing.

Low-risk cases, listed for completeness:

- Repay and liquidation payments at or above the debt mean "repay all"; the pool refunds the excess (`scaling.rs:171-192`).
- A swap-aggregator `referral_id` of `0` means "no referral, no fee" (`execute/mod.rs:89`).
- `transfer_ownership` with `live_until_ledger = 0` cancels a pending transfer, the OpenZeppelin convention (`governance/src/access.rs:59-66`).
- `get_collateral_amount` , `get_borrow_amount` and `get_spoke_usage` return `0` for both "no position" and "empty position".

- Internal only: the Comet adapter passes `i128::MAX` as "no price limit", and the DeFindex adapter uses `0` for "no account bound".

Recommendations

Encode absence in the type: `Option<u64>` for the account id, `Option` for floors and caps that can be disabled, and `Option<i128>` from the health-factor views. `withdraw`'s `to: Option<Address>` and the swap aggregator's rejection of `min_out = 0` already follow this pattern.

Status

[I-08] The rate-model validation lets the cap flatten the curve below optimal utilization

Recommended Action: Fix Code

Description

`calculate_annual_borrow_rate` builds the borrow rate from three segments: below `mid_utilization` the rate rises from `base_borrow_rate` by `slope1`; between `mid` and `optimal_utilization` it rises by a further `slope2`; above `optimal` it rises by `slope3`; the result is then capped at `max_borrow_rate` (`curve.rs:21-59`). The slopes are increments: the rate at `optimal_utilization` is `base + slope1 + slope2`, and the rate at 100% is `base + slope1 + slope2 + slope3`.

`InterestRateModel::verify` checks `base ⩽ slope1 ⩽ slope2 ⩽ slope3 ⩽ max_borrow_rate` and `max_borrow_rate > base` (`types/pool.rs:174-196`). It never relates the sum of the increments to the cap. A model with base 1%, slope1 4%, slope2 6%, slope3 6% and a cap of 6% passes every check, yet the rate reaches the cap one sixth of the way through the second segment and stays at 6% for every utilization above that. The third segment, whose purpose is to make borrowing above `optimal_utilization` expensive enough to pull utilization back, never applies, and `optimal_utilization` loses its meaning. The accrual path and the rate views use the same capped function (`simulate.rs:63`, `views.rs:30-37`), so nothing reports the flattening: a market with a flat top can be listed, and a later `update_params` can flatten a live one.

The monotonicity check also compares increments, not slopes. With `mid` at 10% and `optimal` at 90%, a `slope1` of 2% is 20% per unit of utilization while a `slope2` of 2% is 2.5% per unit, so a curve that passes as non-decreasing is concave across the first kink.

Recommendations

Add to `verify` the constraint the curve's shape depends on: `base + slope1 + slope2 < max_borrow_rate`, so the third segment has room before the cap. Decide whether the cap is a safety bound above the curve, in which case also require `base + slope1 + slope2 + slope3 ⩽ max_borrow_rate`, or the rate at full utilization, in which case require equality; either way, check it. If the curve is meant to steepen at each kink, compare increments divided by segment width rather than the increments themselves.

Status

Future Work

Item	Status

Unofficial

Appendix A: Automated analysis

Every tool run that a conclusion in this report rests on, with its version, configuration, target commit and date. The tools are described in Methodology.

Specula

Run	Dates	Specula version	Target commit	Focus	Candidates	Reproduced
`20261001-121518-e5f5`	2026-10-01 to 10-02	1.2.0	`a2486b25` (v1.1.0)	Owner and admin take-over, governance roles, untrusted callees, price manipulation	22	6
`20261015-114512-af22`	2026-10-15 to 10-16	1.2.0	`a2486b25` (v1.1.0)	Custody of pool, controller and account value	27	4

Each run used Claude Code as its agent at maximum reasoning effort, with the independent review phases disabled, steered by a guidance document that set its goal, scope and questions. The full outputs of both runs, the modeling brief, the TLA+ model, the traces, and the confirmation record and reproduction test of every candidate, were supplied to the XOXNO team rather than reproduced here. The reproduced candidates are under joint triage with XOXNO; those we confirm appear in Findings, and the remaining dispositions are false positives or duplicates of issues XOXNO had already reported.

Plainsight

Date	Plainsight version	Target commit	Scanners	Results
2026-10-02	0.1.0 (`08ce297`)	`a2486b2` (v1.1.0)	9, default rule sets	1,150

The scan covered the whole repository, not only the audited contracts. None of the results falls in a deployed contract: every one that points at a real issue is in off-chain services, tests, Certora tooling or CI configuration, all of which are outside the scope. We did not validate them against the code. We triaged them, and shared the list below with the XOXNO team during the engagement.

Scanner	Results	Real	Disposition
`osv-scanner`	28	26	Advisories in 8 packages; see below. The other 2 are `paste` , unmaintained
`grype`	16	16	Duplicates of the `axios` , `urllib3` and `uuid` advisories
`trivy`	1	1	Duplicate of the `uuid` advisory
`zizmor`	53	50	CI hardening; see below. The other 3 are under `vendor/` , which GitHub never runs
`semgrep`	2	1	Dependabot cooldown, also reported by `zizmor` . The other, a subprocess call built from an argument list, is a false positive
`checkov`	175	1	169 are public contract addresses in `configs/` flagged as high-entropy strings. The real one repeats `zizmor` 's missing `permissions:` ; the rest are style checks on the service Dockerfiles, intended `workflow_dispatch` inputs, and one under `vendor/`
`bandit`	619	0	595 are in `tests/integration` : `assert` , `subprocess` and placeholder strings
`gitleaks`	255	0	Public Stellar contract addresses under a `Token` key in `configs/`
`trufflehog`	1	0	An unverified hex string in a compiled file under `target/` , outside git

No credential was found in the repository.

Dependency advisories

Package	Version → fixed	Lockfile	Advisory
`rustls`	0.23.40, 0.23.42 → 0.23.45	`services/keeper`, `services/lending-exporter`	RUSTSEC-2026-0285
`h2`	0.4.14, 0.4.15 → 0.4.16	`services/keeper`, `services/lending-exporter`	RUSTSEC-2026-0258
`event-listener`	5.4.1 → 5.4.2	`services/keeper`	RUSTSEC-2026-0221
`axios`	1.18.0 → 1.20.0	`tests/integration/sdk`	12 GHSA advisories
`urllib3`	2.7.0 → 2.8.0	`certora/requirements-cli.txt`	PYSEC-2026-4175, -4176, -4177
`uuid`	8.3.2 → 11.1.1	`tests/integration/sdk`	GHSA-w5hq-g745-h8pq
`anyhow`	1.0.102 → 1.0.103	root, `services/keeper`	RUSTSEC-2026-0190
`paste`	1.0.15, no fix	root, `tests/fuzz`	RUSTSEC-2024-0436 (unmaintained)

`rustls`, `h2` and `event-listener` are the only ones in code that runs in production: the keeper, which submits transactions, and the lending exporter. `axios`, `uuid` and `urllib3` run only in CI and on developer machines. `anyhow` and `paste` are build-time dependencies and are not compiled into the contracts.

CI configuration

`zizmor` rule	Results	Where
`artipacked`	26	Every `actions/checkout` in 11 workflows keeps the token in `.git/config`
`excessive-permissions`	14	`fuzz.yml` and `certora-verification.yml` have no `permissions:` key
`template-injection`	7	`matrix.*` expressions in `certora-verification.yml`, from the repository's own `profiles.json`, on `workflow_dispatch` only
`cache-poisoning`	2	`setup-node` in `release.yml`
`dependabot-cooldown`	1	`.github/dependabot.yml`